



كلية العلوم
والتقنيات – مراكش
FACULTÉ DES SCIENCES ET
TECHNIQUE – MARRAKECH

UNIVERSITÉ CADI AYYAD - MARRAKECH
FACULTÉ DES SCIENCES ET TECHNIQUES ANNÉE
2025-2026

CONTRÔLE CONTINU : SÉCURITÉ DES SYSTÈMES ET
RÉSEAUX

Conception et sécurisation d'une
architecture Entreprise avec
OpenVPN, pfSense et Suricata

Élève ingénieur :
BENDER SALMA

Enseignant :
Charafeddine AITZAOUAT

28 mai 2026

Table des matières

1	Introduction	3
2	Partie 1 : Mise en place de l'architecture	3
2.1	Objectif	3
2.2	Machines virtuelles utilisées	3
2.3	Configuration réseau	4
2.4	Plan d'adressage	8
2.5	Architecture réseau	8
2.6	Description de l'architecture	8
2.7	Configuration pfSense	9
2.8	Configuration Ubuntu Server	10
2.9	Tests de connectivité	11
2.10	Configuration Windows 11	12
2.11	Configuration Kali Linux	13
2.12	Récapitulatif du plan d'adressage	13
2.13	Conclusion de la Phase 1	14
3	Déploiement OpenVPN	15
3.1	Étape 1 Création de l'autorité de certification (CA)	15
3.2	Étape 2 Création du certificat serveur	16
3.3	Étape 3 Création de l'utilisateur VPN	17
3.4	Étape 4 Configuration du serveur OpenVPN	18
3.5	Étape 5 Configuration des règles de pare-feu	19
3.6	Étape 6 Export et installation du client OpenVPN	21
3.7	Étape 7 Connexion et tests VPN	22
3.8	Étape 8 Activation des services sur Ubuntu Server	25
3.8.1	Test d'accès SSH	26
3.8.2	Test d'accès HTTP	27
3.9	Conclusion de la Phase 2	28
4	Partie 3 Durcissement du serveur Ubuntu	28
4.1	Étape 1 Mise à jour du système	29
4.2	Étape 2 Suppression des services inutiles	29
4.3	Étape 3 Désactivation d'IPv6	31
4.4	Étape 4 Durcissement du noyau via sysctl	32
4.5	Étape 5 Configuration du pare-feu UFW	34
4.6	Étape 6 Verrouillage du compte root	36
4.7	Étape 7 Protection du chargeur de démarrage GRUB	37
5	Sécurisation PAM	38
5.1	Expiration des mots de passe	38
5.2	Complexité des mots de passe	38
5.3	Verrouillage après tentatives échouées	39
6	Sécurisation SSH	39

7	Test de la protection GRUB sur Ubuntu	40
8	Partie 4 Services sur Ubuntu	43
8.1	Vérification des services	43
8.2	Test de connectivité Ping depuis Windows	43
8.3	Test d'accès SSH depuis Windows	45
8.4	Test d'accès HTTP depuis Windows	45
9	Partie 5 Déploiement du système IDS/IPS Suricata	46
9.1	Étape 1 Installation du paquet Suricata	47
9.2	Étape 2 Configuration globale du moteur	47
9.3	Étape 3 Synchronisation et mise à jour des signatures	48
9.4	Étape 4 Configuration de l'interface WAN	49
9.5	Étape 5 Sélection des catégories de règles de détection	50
9.6	Étape 6 Implémentation des règles personnalisées (<i>Custom Rules</i>)	50
9.7	Étape 7 Initialisation du service et vérification	52
9.8	Étape 8 Observation de l'état initial des alertes	53
9.9	Bilan d'avancement de l'architecture	53
10	Phase 6 Attaques depuis Kali Linux	54
10.1	Contexte	54
10.2	Attaque 1 Scan réseau SYN (Nmap -sS)	54
10.3	Attaque 2 Détection de services (Nmap -sV)	55
10.4	Attaque 3 Ping Flood ICMP	56
10.5	Attaque 4 Brute Force SSH (Hydra)	57
10.6	Bilan des attaques détectées	58
11	Phase 7 Firewalling	58
11.1	Objectif	58
11.2	Règle 1 Blocage ICMP sur WAN	58
11.3	Règle 2 Autoriser HTTP via VPN uniquement	59
11.4	Règle 3 Autoriser SSH via VPN uniquement	60
11.5	Règle 4 Blocage total de la machine Kali	60
11.6	Tests de validation	61
11.6.1	Test 1 Kali Linux bloquée	61
11.6.2	Test 2 Accès refusé sans VPN	62
11.6.3	Test 3 Accès autorisé avec VPN	63
11.7	Bilan du Firewalling	64
12	Conclusion	64
12.1	Bilan des réalisations	64
12.2	Compétences acquises	65
12.3	Enseignements et perspectives	65

1 Introduction

Avec laugmentation des cyberattaques et des accès distants dans les entreprises modernes, la sécurisation des infrastructures réseau devient une nécessité essentielle. Les organisations doivent mettre en place des mécanismes capables de protéger les ressources internes, contrôler les accès distants et détecter les activités malveillantes en temps réel.

Dans ce projet, nous avons conçu une architecture réseau sécurisée basée sur plusieurs technologies de cybersécurité, notamment **pfSense**, **OpenVPN** et **Suricata IDS/IPS**. Cette infrastructure permet de :

- sécuriser les communications distantes grâce à un VPN chiffré ;
- protéger un serveur Ubuntu interne critique ;
- détecter les attaques réseau et comportements suspects ;
- appliquer des politiques de filtrage et de contrôle d'accès.

L'environnement a été virtualisé sous **VirtualBox** afin de simuler une infrastructure d'entreprise réaliste tout en limitant la consommation des ressources matérielles. L'architecture est composée de quatre machines virtuelles :

- un pare-feu **pfSense** jouant le rôle de firewall, serveur VPN et IDS/IPS ;
- un **serveur Ubuntu** représentant le serveur interne de l'entreprise ;
- un client **Windows** utilisé pour l'accès distant via VPN ;
- une machine **Kali Linux** utilisée pour simuler des attaques.

Ce projet a pour objectif principal de mettre en œuvre une architecture sécurisée capable de résister à différentes attaques réseau tout en assurant la disponibilité et la confidentialité des services internes.

2 Partie 1 : Mise en place de l'architecture

2.1 Objectif

La première étape du projet consiste à mettre en place l'environnement virtuel et à préparer l'architecture réseau qui servira de base aux différentes phases de sécurisation.

Cette phase comprend :

- l'installation des machines virtuelles ;
- la configuration des interfaces réseau ;
- la mise en place du plan d'adressage ;
- la vérification de la connectivité entre les équipements.

2.2 Machines virtuelles utilisées

L'infrastructure est composée des machines suivantes :

Machine	Système	Rôle
pfSense	pfSense CE	Firewall + VPN + IDS/IPS
Ubuntu Server	Ubuntu Server	Serveur interne sécurisé
Windows Client	Windows 10	Client VPN distant
Kali Linux	Kali Linux	Machine attaquante

TABLE 1 – Machines virtuelles utilisées

2.3 Configuration réseau

Deux types de réseaux VirtualBox ont été utilisés :

- **Bridged Adapter** : permet aux machines d'accéder au réseau externe ;
- **Host-Only Adapter** : utilisé pour créer le réseau LAN interne sécurisé.

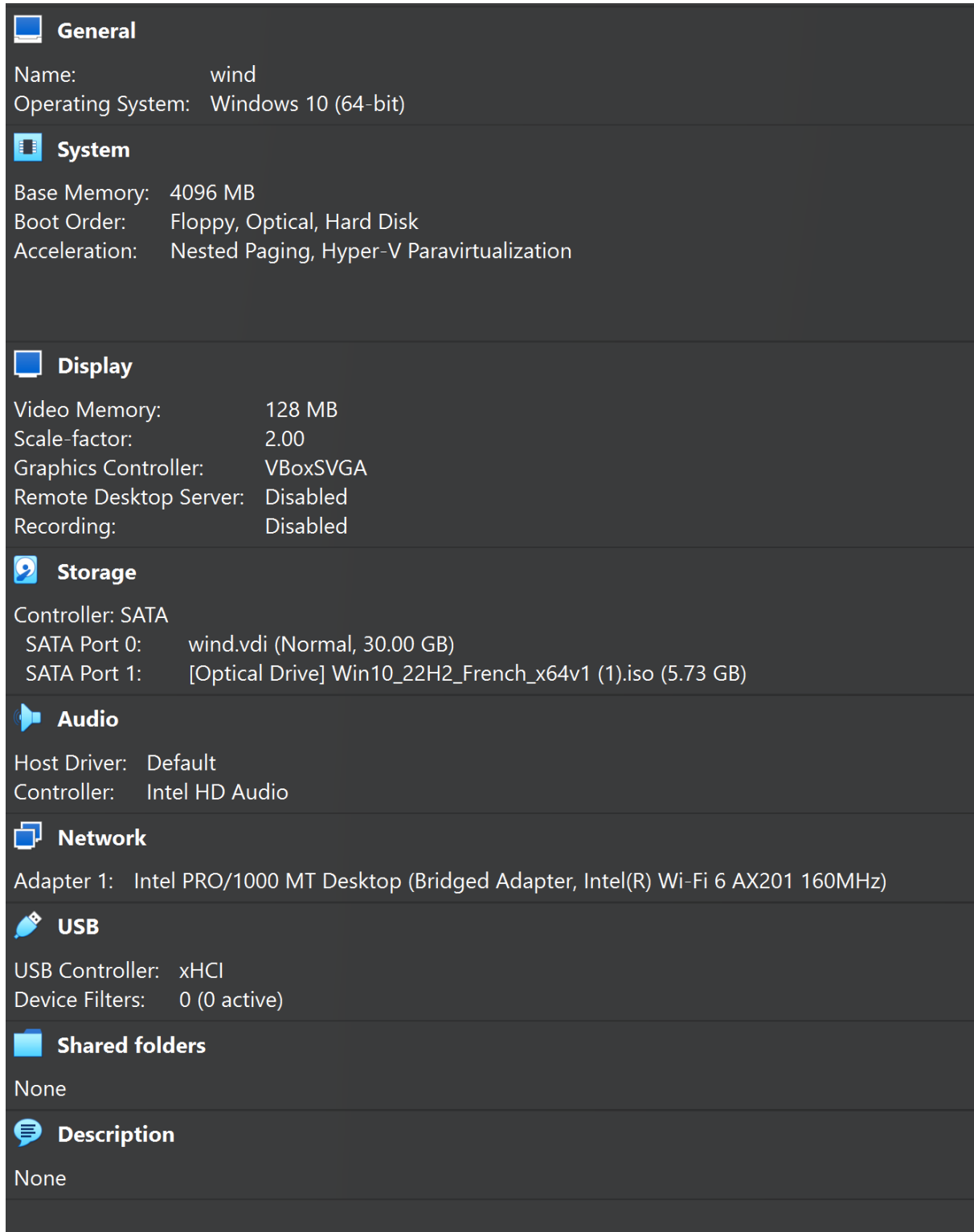


FIGURE 1 – Installation et configuration de Windows 10


General

Name: pfSense

Operating System: FreeBSD (64-bit)


System

Base Memory: 4096 MB

Processors: 2

Boot Order: Floppy, Optical, Hard Disk

Acceleration: Nested Paging


Display

Video Memory: 16 MB

Scale-factor: 2.75

Graphics Controller: VMSVGA

Remote Desktop Server: Disabled

Recording: Disabled


Storage

Controller: IDE

IDE Primary Device 0: pfSense.vdi (Normal, 30.61 GB)

IDE Secondary Device 0: [Optical Drive] Empty


Audio

Host Driver: Default

Controller: ICH AC97


Network

Adapter 1: Intel PRO/1000 MT Desktop (Bridged Adapter, Intel(R) Ethernet Connection (10) I219-V)

Adapter 2: Intel PRO/1000 MT Desktop (Host-only Adapter, 'VirtualBox Host-Only Ethernet Adapter')


USB

USB Controller: OHCI, EHCI

Device Filters: 0 (0 active)


Shared folders

None


Description

None

FIGURE 2 – Installation et configuration de pfSense CE

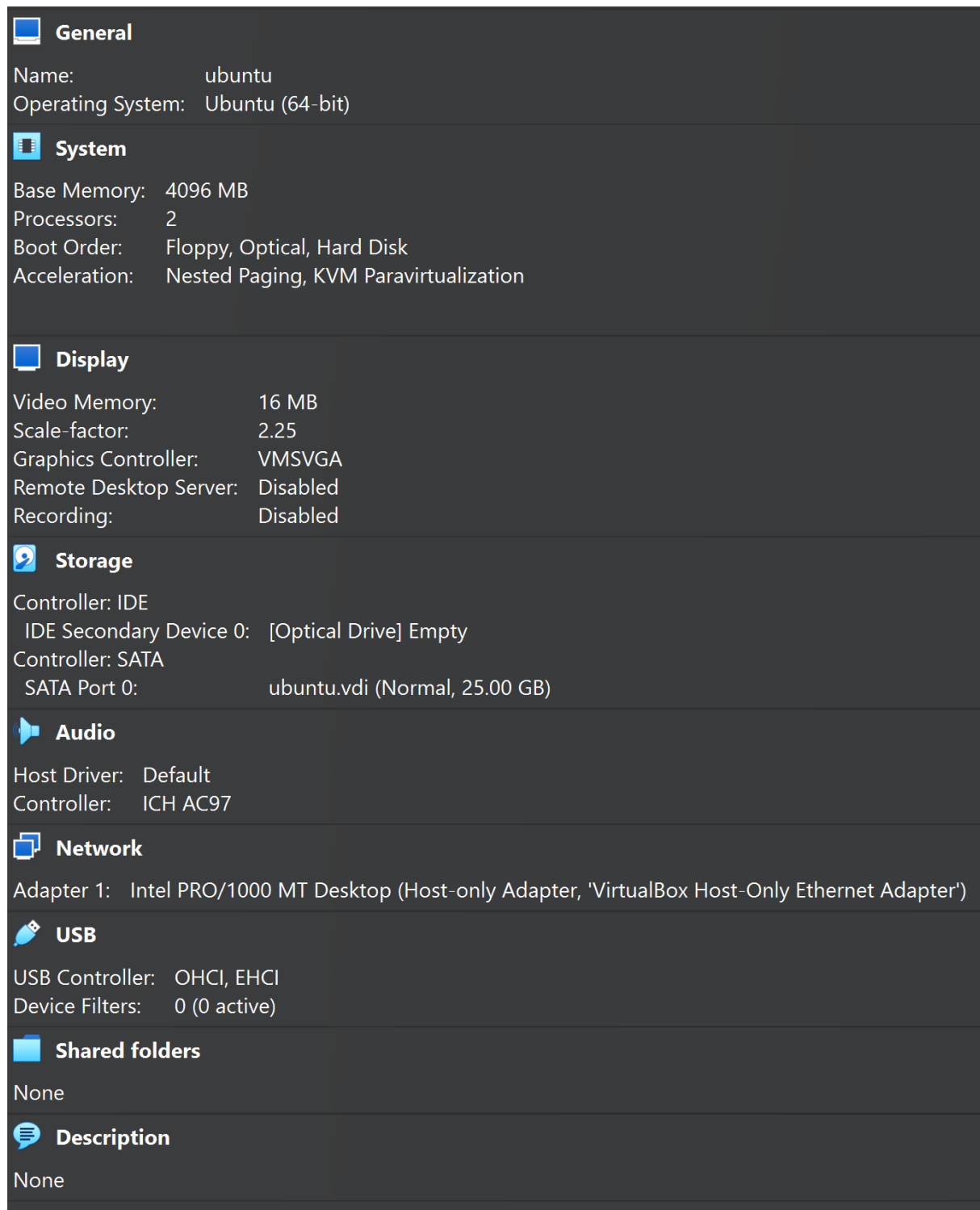


FIGURE 3 – Installation et configuration de Ubuntu


General

Name: kali_api
Operating System: Ubuntu (64-bit)


System

Base Memory: 4096 MB
Processors: 2
Boot Order: Floppy, Optical, Hard Disk
Acceleration: Nested Paging, KVM Paravirtualization


Display

Video Memory: 16 MB
Scale-factor: 2.25
Graphics Controller: VMSVGA
Remote Desktop Server: Disabled
Recording: Disabled


Storage

Controller: IDE
IDE Secondary Device 0: [Optical Drive] Empty
Controller: SATA
SATA Port 0: kali_api.vdi (Normal, 25.00 GB)


Audio

Host Driver: Default
Controller: ICH AC97


Network

Adapter 1: Intel PRO/1000 MT Desktop (Bridged Adapter, Intel(R) Wi-Fi 6 AX201 160MHz)


USB

USB Controller: OHCI, EHCI
Device Filters: 0 (0 active)


Shared folders

None


Description

None

FIGURE 4 – Installation et configuration de kali

2.4 Plan d'adressage

Machine	Adresse IP
pfSense LAN	192.168.56.1
Ubuntu Server	192.168.56.10
Réseau VPN	10.8.0.0/24

TABLE 2 – Plan d'adressage IP

2.5 Architecture réseau

La figure suivante représente l'architecture globale mise en place dans ce projet.

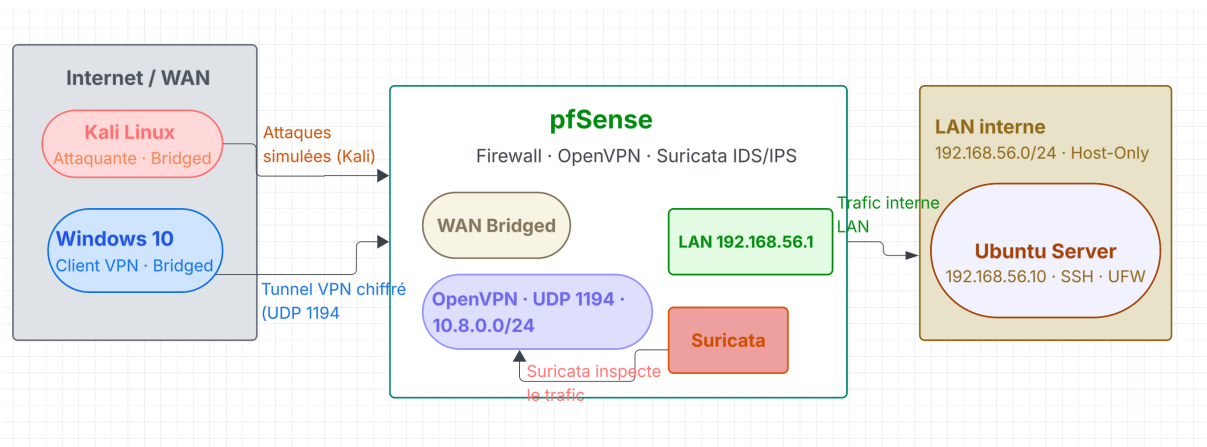


FIGURE 5 – Architecture sécurisée avec pfSense, OpenVPN et Suricata

2.6 Description de l'architecture

L'architecture est organisée autour du firewall **pfSense**, qui constitue l'élément central de sécurité.

- L'interface **WAN** de pfSense est connectée au réseau externe via le mode Bridged.
- L'interface **LAN** de pfSense est connectée au réseau interne 192.168.56.0/24.
- Le serveur Ubuntu héberge les services internes accessibles uniquement selon les règles de sécurité définies.
- Le client Windows se connecte à distance via un tunnel **OpenVPN** sécurisé utilisant le protocole UDP sur le port 1194.
- La machine Kali Linux est utilisée pour simuler plusieurs attaques réseau afin de tester les mécanismes de détection et de protection.
- **Suricata IDS/IPS** analyse le trafic réseau afin de détecter les scans, attaques SSH, floods ICMP et autres comportements suspects.

2.7 Configuration pfSense

```
Starting CRON... done.
Starting package OpenVPN Client Export Utility...done.
pfSense 2.8.1-RELEASE amd64 20251215-1731
Bootup complete

FreeBSD/amd64 (pfSense.home.arpa) (ttyv0)

VirtualBox Virtual Machine - Netgate Device ID: 76ba1a57b292505dbcba

*** Welcome to pfSense 2.8.1-RELEASE (amd64) on pfSense ***

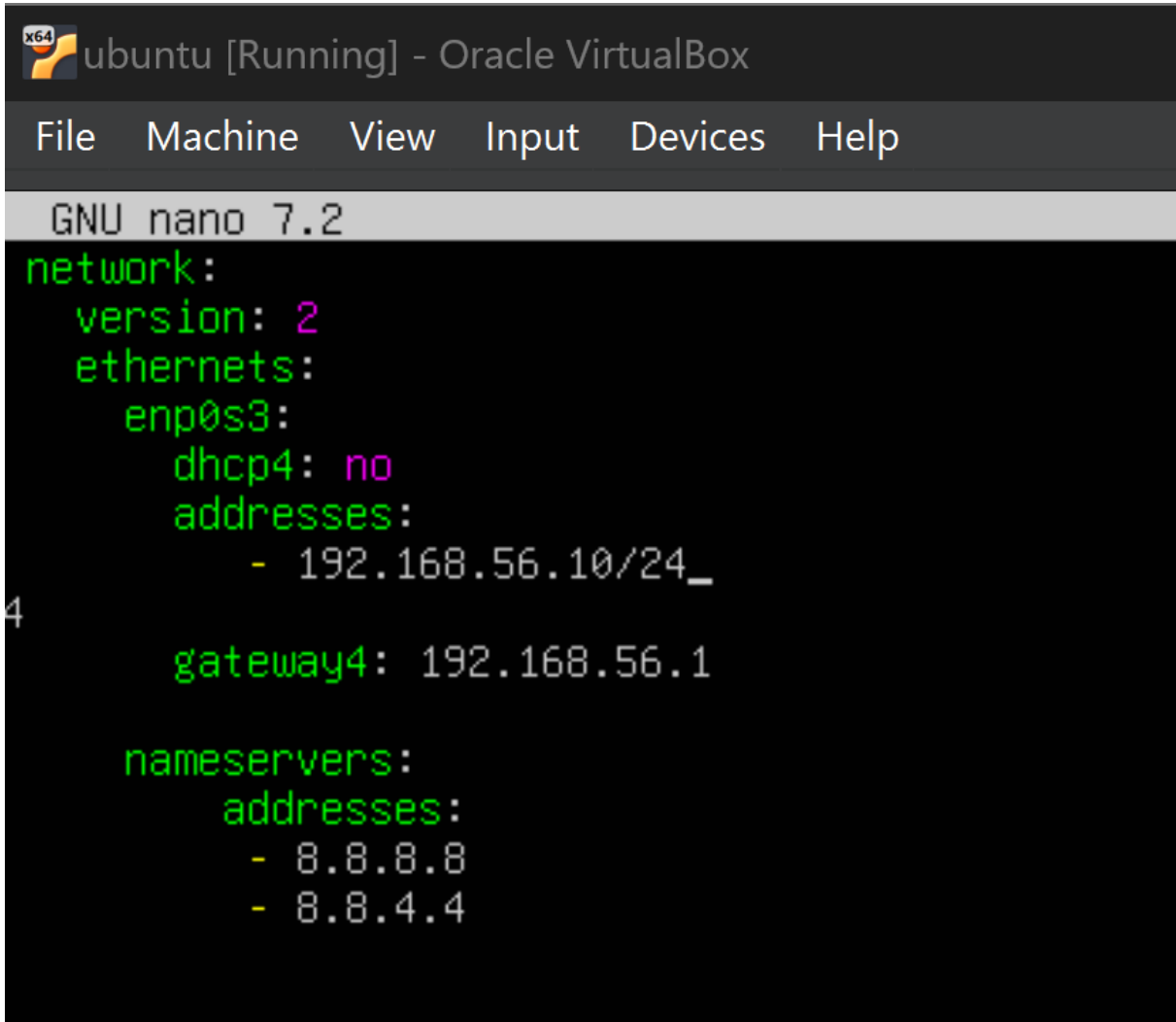
WAN (wan) -> em0 -> v4/DHCP4: 192.168.100.77/24
LAN (lan) -> em1 -> v4: 192.168.56.1/24

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                10) Filter Logs
2) Set interface(s) IP address     11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                   14) Enable Secure Shell (sshd)
6) Halt system                     15) Restore recent configuration
7) Ping host                       16) Restart PHP-FPM
8) Shell

Enter an option: █
```

FIGURE 6 – Adresse ip de pfsens

2.8 Configuration Ubuntu Server

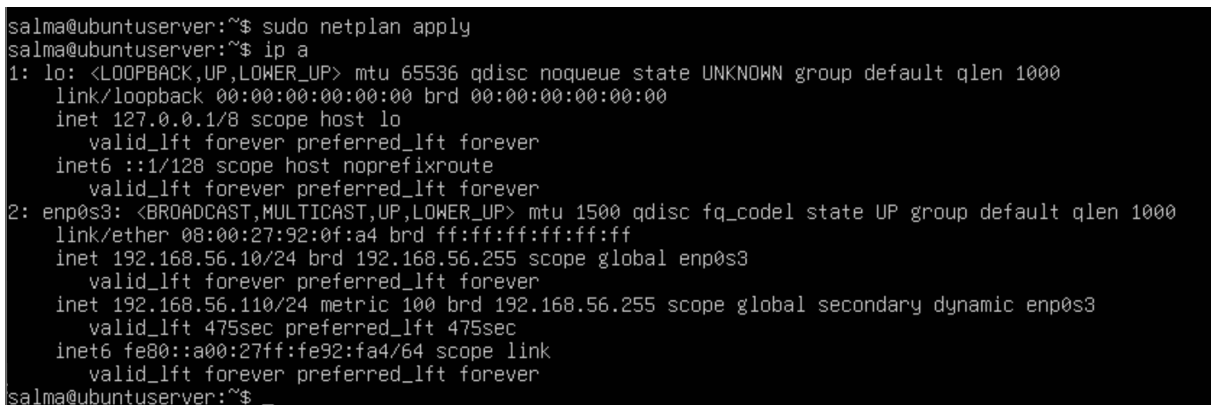


```

x64 ubuntu [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 7.2
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.10/24_
4
      gateway4: 192.168.56.1

  nameservers:
    addresses:
      - 8.8.8.8
      - 8.8.4.4
    
```

FIGURE 7 – Configuration d'une l'IP statique



```

salma@ubuntuserver:~$ sudo netplan apply
salma@ubuntuserver:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:92:0f:a4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet 192.168.56.110/24 metric 100 brd 192.168.56.255 scope global secondary dynamic enp0s3
        valid_lft 475sec preferred_lft 475sec
    inet6 fe80::a00:27ff:fe92:fa4/64 scope link
        valid_lft forever preferred_lft forever
salma@ubuntuserver:~$ _
    
```

FIGURE 8 – Application de la configuration et la vérification de l'adresse ip

2.9 Tests de connectivité

Après la configuration des interfaces réseau et des adresses IP, plusieurs tests de connectivité ont été réalisés :

- ping entre pfSense et Ubuntu Server ;
- vérification de l'accès réseau du client Windows ;
- validation de la communication entre les machines virtuelles ;
- test d'accès Internet via l'interface WAN.

Les résultats obtenus confirment le bon fonctionnement de l'architecture réseau mise en place.

```

WAN (wan) -> em0 -> v4/DHCP4: 192.168.100.77/24
LAN (lan) -> em1 -> v4: 192.168.56.1/24

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                10) Filter Logs
2) Set interface(s) IP address      11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                   14) Enable Secure Shell (sshd)
6) Halt system                     15) Restore recent configuration
7) Ping host                       16) Restart PHP-FPM
8) Shell

Enter an option: 8

[2.8.1-RELEASE][root@pfSense.home.arpal]/root: ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10): 56 data bytes
64 bytes from 192.168.56.10: icmp_seq=0 ttl=64 time=2.450 ms
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=2.601 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=2.568 ms
^C
--- 192.168.56.10 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 2.450/2.540/2.601/0.065 ms
[2.8.1-RELEASE][root@pfSense.home.arpal]/root:
    
```

FIGURE 9 – pfsense ping Ubuntu

```

salma@salma:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=64 time=1.80 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=64 time=2.05 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=64 time=2.31 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=64 time=3.82 ms
^C
--- 192.168.56.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 1.799/2.494/3.817/0.784 ms
salma@salma:~$
    
```

FIGURE 10 – Ubuntu ping pfsense

```

salma@salma:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=68.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=31.1 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 31.144/49.826/68.508/18.682 ms
salma@salma:~$ ping google.com
PING google.com (216.58.206.14) 56(84) bytes of data.
64 bytes from fra16s20-in-f14.1e100.net (216.58.206.14): icmp_seq=1 ttl=115 time=34.6 ms
64 bytes from fra16s20-in-f14.1e100.net (216.58.206.14): icmp_seq=2 ttl=115 time=31.1 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 31.136/32.878/34.621/1.742 ms
salma@salma:~$ _
    
```

FIGURE 11 – Ubuntu ping Internet

2.10 Configuration Windows 11

Windows 11 est configuré en mode *Bridged Adapter*, ce qui lui permet d'obtenir automatiquement une adresse IP depuis le routeur Wi-Fi via DHCP, dans le même sous-réseau que l'interface WAN de pfSense. Cette machine jouera le rôle de client distant qui se connectera au réseau interne de l'entreprise via le tunnel VPN OpenVPN, simulant ainsi un employé travaillant à distance de manière sécurisée.

```

C:\> Invite de commandes

Suffixe DNS propre à la connexion. . . . :
Adresse IPv6 de liaison locale. . . . . : fe80::8637:92b0:efe1:509a%6
Adresse IPv4. . . . . : 192.168.100.82
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . : 192.168.100.1

C:\Users\salma>ping 192.168.100.1

Envoi d'une requête 'Ping' 192.168.100.1 avec 32 octets de données :
Réponse de 192.168.100.1 : octets=32 temps=322 ms TTL=64
Réponse de 192.168.100.1 : octets=32 temps=3 ms TTL=64

Statistiques Ping pour 192.168.100.1:
    Paquets : envoyés = 2, reçus = 2, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 3ms, Maximum = 322ms, Moyenne = 162ms
Ctrl+C
^C
C:\Users\salma>ping 8.8.8.8

Envoi d'une requête 'Ping' 8.8.8.8 avec 32 octets de données :
Réponse de 8.8.8.8 : octets=32 temps=61 ms TTL=116

Statistiques Ping pour 8.8.8.8:
    Paquets : envoyés = 1, reçus = 1, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 61ms, Maximum = 61ms, Moyenne = 61ms
Ctrl+C
^C
C:\Users\salma>_
    
```

FIGURE 12 – ip de windows et le teste d'Internet

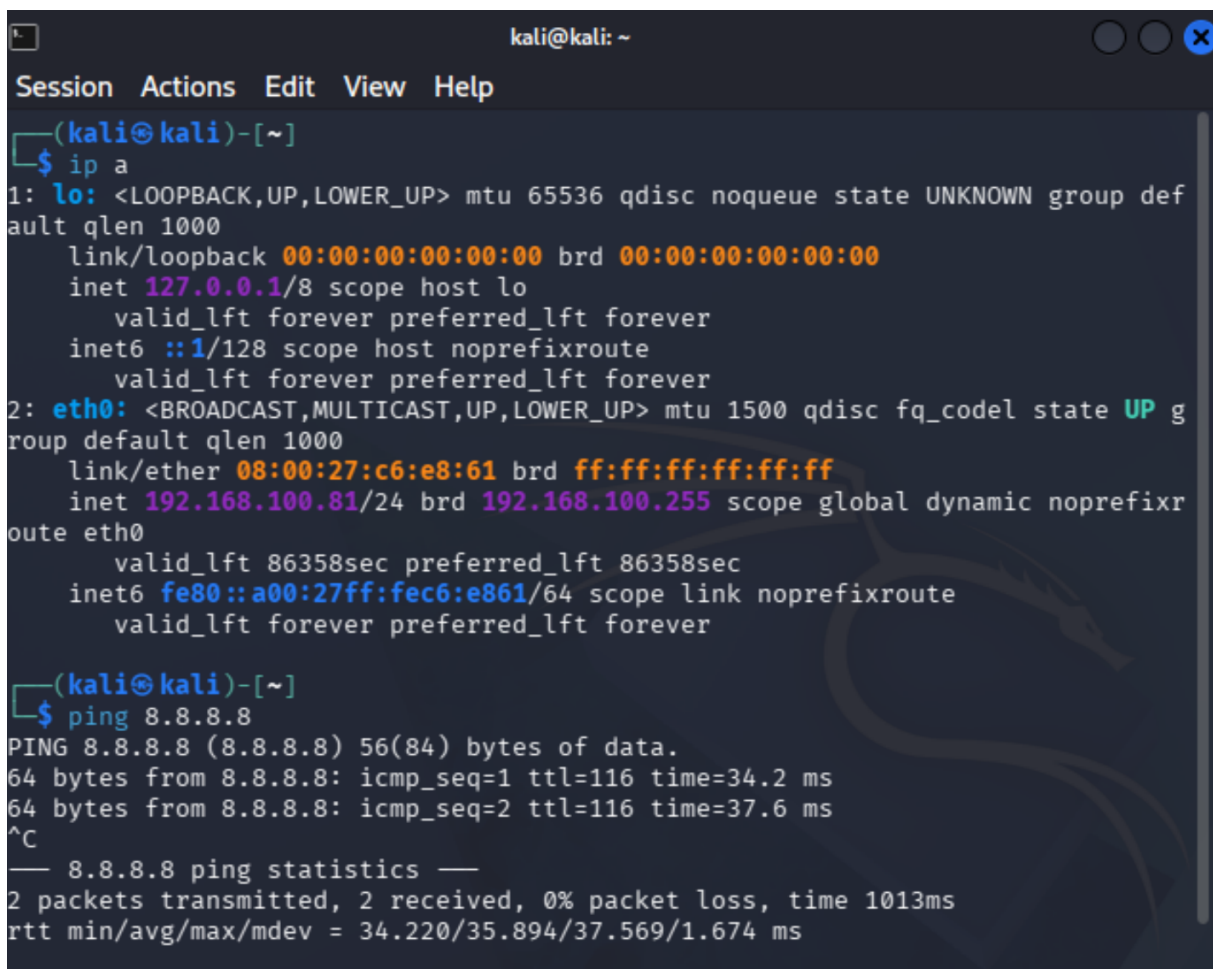
```

[2.8.1-RELEASE][root@pfSense.home.arpal/root: ping 192.168.31.165
PING 192.168.31.165 (192.168.31.165): 56 data bytes
^C
--- 192.168.31.165 ping statistics ---
6 packets transmitted, 0 packets received, 100.0% packet loss
[2.8.1-RELEASE][root@pfSense.home.arpal/root:
    
```

FIGURE 13 – failed communication between pfsense vers windows 11

2.11 Configuration Kali Linux

Kali Linux est configuré en mode *Bridged Adapter*, ce qui lui permet d'obtenir automatiquement une adresse IP depuis le routeur Wi-Fi via DHCP, dans le même sous-réseau que l'interface WAN de pfSense. Cette machine jouera le rôle d'attaquant externe simulant des menaces provenant d'Internet.



```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:c6:e8:61 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.81/24 brd 192.168.100.255 scope global dynamic noprefixr
oute eth0
        valid_lft 86358sec preferred_lft 86358sec
    inet6 fe80::a00:27ff:fec6:e861/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali@kali)-[~]
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=34.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=37.6 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1013ms
rtt min/avg/max/mdev = 34.220/35.894/37.569/1.674 ms
    
```

FIGURE 14 – Adresse IP de Kali Linux et test de connectivité Internet

2.12 Récapitulatif du plan d'adressage

Le tableau suivant résume l'ensemble des adresses IP attribuées aux machines virtuelles de l'infrastructure :

Machine	Rôle	Interface VirtualBox	Adresse IP	Réseau
pfSense (WAN)	Firewall / VPN	Bridged (Wi-Fi)	192.168.100.80	192.168.100.0/24
pfSense (LAN)	Firewall / VPN	Host-Only	192.168.56.1	192.168.56.0/24
Ubuntu Server	Serveur interne	Host-Only	192.168.56.10	192.168.56.0/24
Windows Client	Client VPN	Bridged (Wi-Fi)	192.168.100.82	192.168.100.0/24
Kali Linux	Machine attaquante	Bridged (Wi-Fi)	192.168.100.81	192.168.100.0/24
VPN (tunnel)	OpenVPN	–	10.8.0.0/24	10.8.0.0/24

TABLE 3 – Plan d'adressage complet de l'infrastructure

2.13 Conclusion de la Phase 1

L'ensemble des machines virtuelles ont été installées et configurées avec succès. La connectivité entre les différentes zones du réseau a été validée :

- pfSense assure le routage entre le réseau LAN (192.168.56.0/24) et Internet via le WAN (192.168.100.80) ;
- Ubuntu Server est accessible depuis pfSense sur l'adresse 192.168.56.10 ;
- Windows Client et Kali Linux disposent d'un accès Internet direct via le réseau Wi-Fi 192.168.100.0/24 ;
- Le réseau VPN 10.8.0.0/24 sera configuré à la phase suivante pour permettre l'accès distant sécurisé.

3 Déploiement OpenVPN

Cette phase décrit la mise en place d'un accès distant sécurisé par tunnel VPN de type Client-to-Site entre le client Windows et le réseau interne de l'entreprise, en utilisant OpenVPN intégré à pfSense. L'objectif est de permettre à un employé distant de se connecter au réseau interne de manière chiffrée et authentifiée.

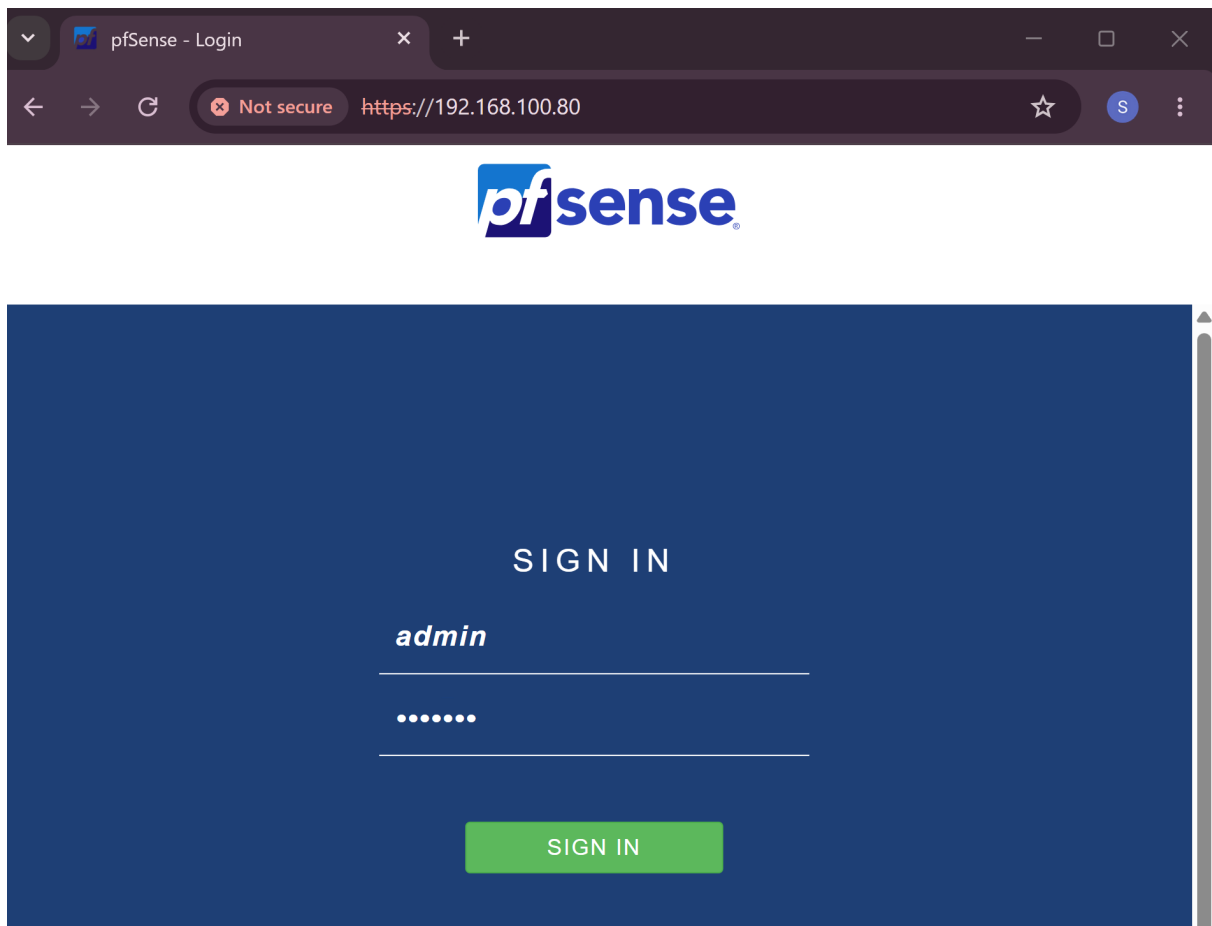


FIGURE 15 – Accéder à l'interface web pfSense

L'interface web de pfSense est accessible depuis le réseau local à l'adresse `http://192.168.100.80`. Elle permet de configurer l'ensemble des services du pare-feu, notamment OpenVPN, les certificats et les règles de filtrage.

3.1 Étape 1 Création de l'autorité de certification (CA)

La première étape consiste à créer une autorité de certification interne (CA) qui sera utilisée pour signer les certificats du serveur et des clients VPN. Cette CA garantit l'authenticité des entités qui se connectent au VPN et constitue la racine de confiance de l'infrastructure PKI.

La CA est créée avec les paramètres suivants :

- **Nom** : VPN-CA
- **Méthode** : Create an internal Certificate Authority
- **Algorithme** : RSA 2048 bits

- Digest : SHA256
- Durée de validité : 3650 jours (10 ans)

Authorities

Certificates

Revocation

Search

Search term

Both

Search

Clear

Enter a search string or *nix regular expression to search certificate names and distinguished names.

Certificate Authorities

Name	Internal	Issuer	Certificates	Distinguished Name	In Use	Actions
VPN-CA	✓	self-signed	0	CN=internal-ca Valid From: Sun, 24 May 2026 01:30:33 +0000 Valid Until: Wed, 21 May 2036 01:30:33 +0000		

+ Add

FIGURE 16 – Création de l'autorité de certification VPN-CA

3.2 Étape 2 Création du certificat serveur

Un certificat serveur est ensuite généré et signé par la CA précédemment créée. Ce certificat est utilisé par le serveur OpenVPN pour s'authentifier auprès des clients lors de l'établissement du tunnel TLS. Il est de type *Server Certificate*, ce qui lui confère les extensions nécessaires pour l'authentification serveur.

Authorities

Certificates

Revocation

Search

Search term

Both

Search

Clear

Enter a search string or *nix regular expression to search certificate names and distinguished names.

Certificates

Name	Issuer	Distinguished Name	In Use	Actions
GUI default (6a123831cc951) Server Certificate CA: No Server: Yes	self-signed	O=pfSense GUI default Self-Signed Certificate, CN=pfSense-6a123831cc951 Valid From: Sat, 23 May 2026 23:28:50 +0000 Valid Until: Fri, 25 Jun 2027 23:28:50 +0000	webConfigurator	    
VPN-Server-Cert Server Certificate CA: No Server: Yes	VPN-CA	CN=vpn-server Valid From: Sun, 24 May 2026 01:38:10 +0000 Valid Until: Wed, 21 May 2036 01:38:10 +0000		    

+ Add/Sign

FIGURE 17 – Création du certificat serveur VPN-Server-Cert

3.3 Étape 3 Création de l'utilisateur VPN

Un compte utilisateur `vpnuser` est créé dans le gestionnaire d'utilisateurs de pfSense. Cet utilisateur représente l'employé distant qui se connectera au VPN. Un certificat client lui est associé, signé par la CA interne. L'authentification VPN repose ainsi sur deux facteurs : le certificat client et le mot de passe, ce qui renforce considérablement la sécurité de l'accès distant.

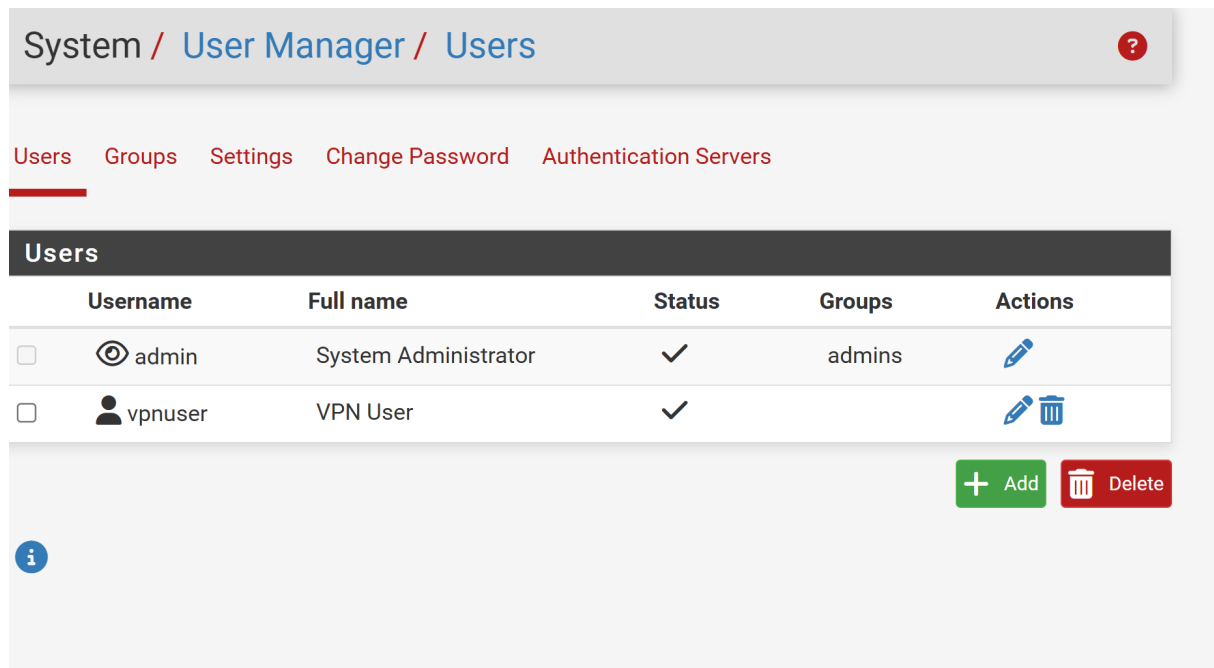


FIGURE 18 – Utilisateur VPN créé avec succès

3.4 Étape 4 Configuration du serveur OpenVPN

Le serveur OpenVPN est configuré en mode *Remote Access (SSL/TLS + User Auth)*, ce qui impose à la fois une authentification par certificat et par identifiants. Les paramètres principaux sont les suivants :

- **Protocole** : UDP sur IPv4, port 1194
- **Interface** : WAN
- **Chiffrement** : AES-256-CBC
- **Digest** : SHA256
- **Paramètres DH** : 2048 bits
- **Réseau tunnel** : 10.8.0.0/24
- **Réseau local** : 192.168.56.0/24
- **DNS** : 8.8.8.8 / 8.8.4.4

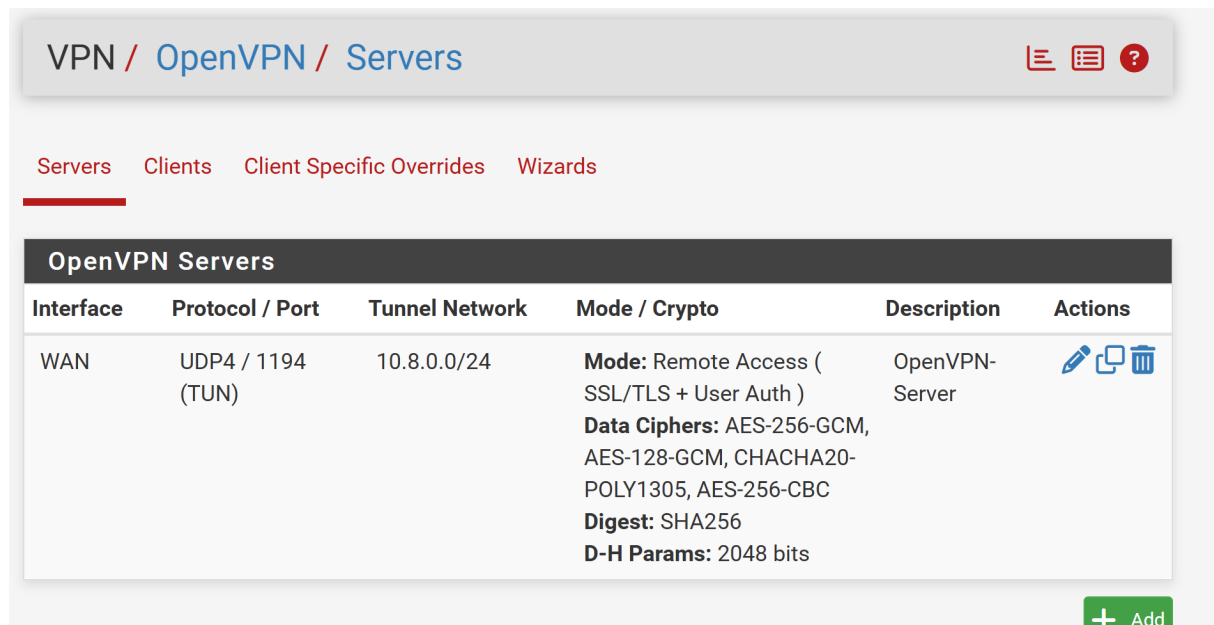


FIGURE 19 – Serveur OpenVPN configuré avec succès

3.5 Étape 5 Configuration des règles de pare-feu

- Deux règles de pare-feu sont nécessaires pour permettre le fonctionnement du VPN :
- **Règle WAN** : autorise le trafic UDP entrant sur le port 1194 vers l'adresse WAN de pfSense, permettant aux clients distants d'initier la connexion VPN.
 - **Règle OpenVPN** : autorise tout le trafic provenant des clients connectés au VPN vers le réseau interne, permettant l'accès aux services hébergés sur Ubuntu Server.

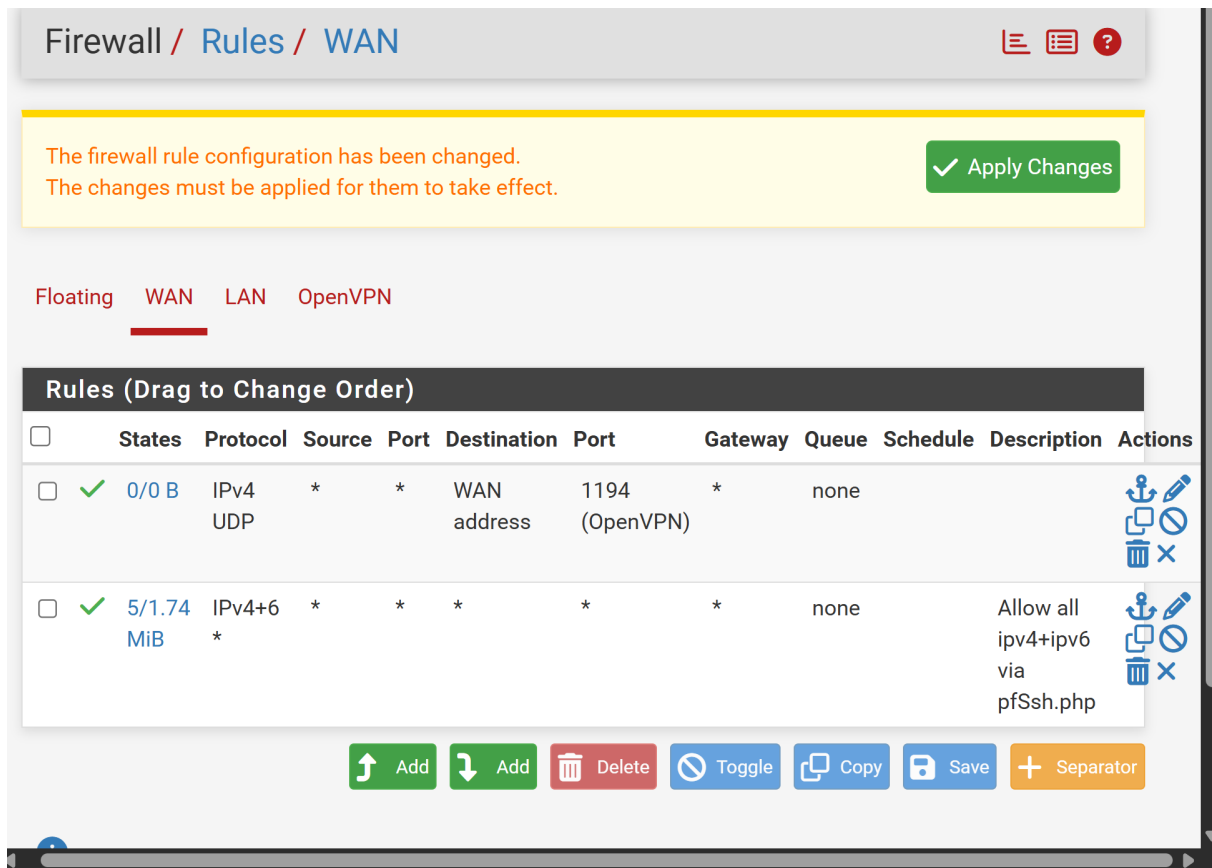


FIGURE 20 – Règle WAN autorisation du port UDP 1194

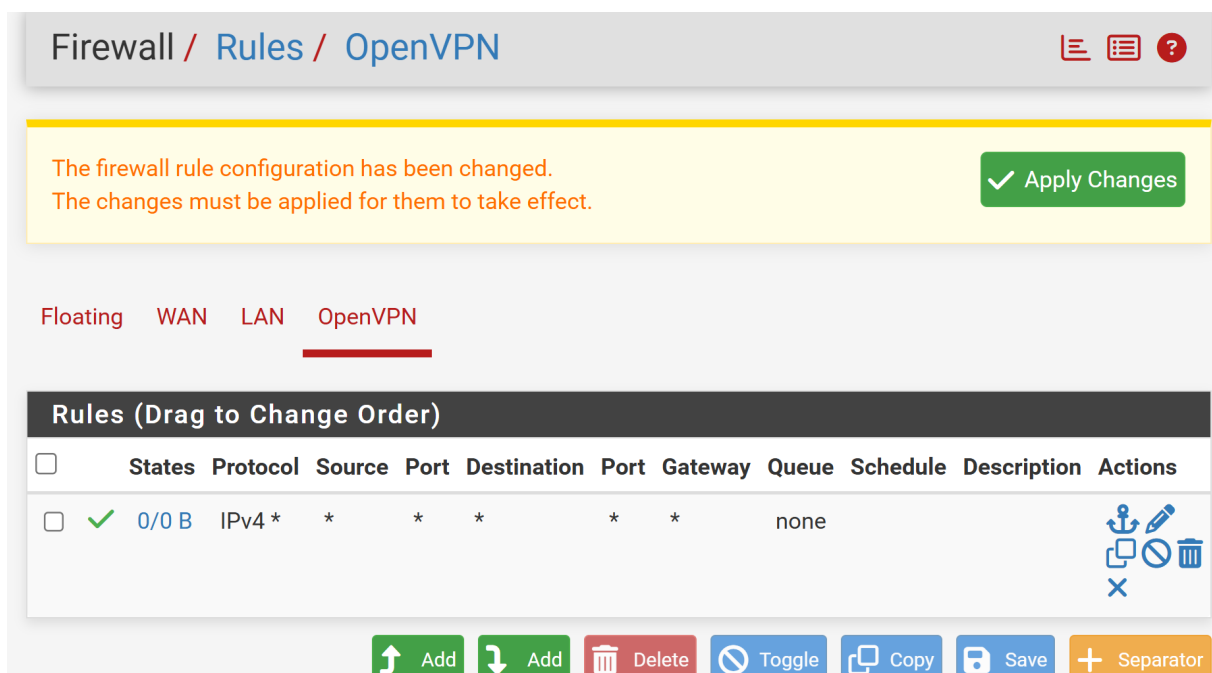


FIGURE 21 – Règle OpenVPN autorisation du trafic VPN

3.6 Étape 6 Export et installation du client OpenVPN

Le package `openvpn-client-export` est installé sur pfSense afin de générer automatiquement le fichier de configuration client `.ovpn`. Ce fichier contient l'ensemble des paramètres nécessaires à la connexion : adresse du serveur, port, certificats, clé TLS et configuration du tunnel.

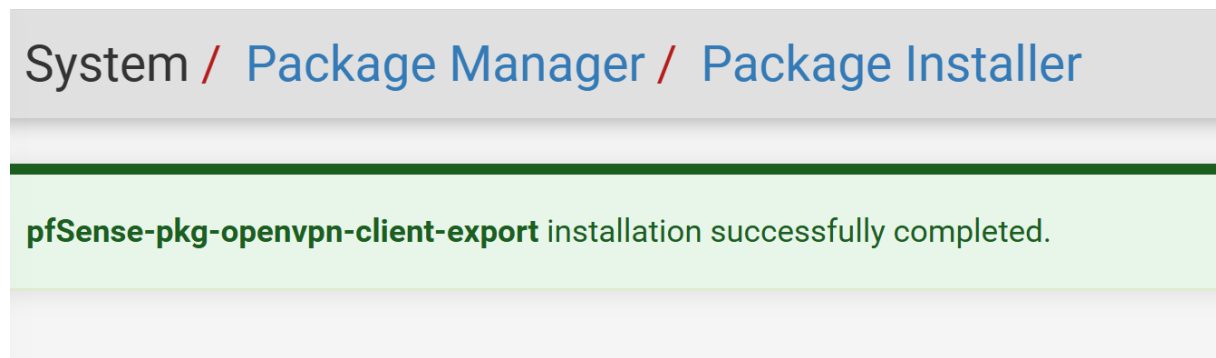


FIGURE 22 – Package openvpn-client-export installé avec succès

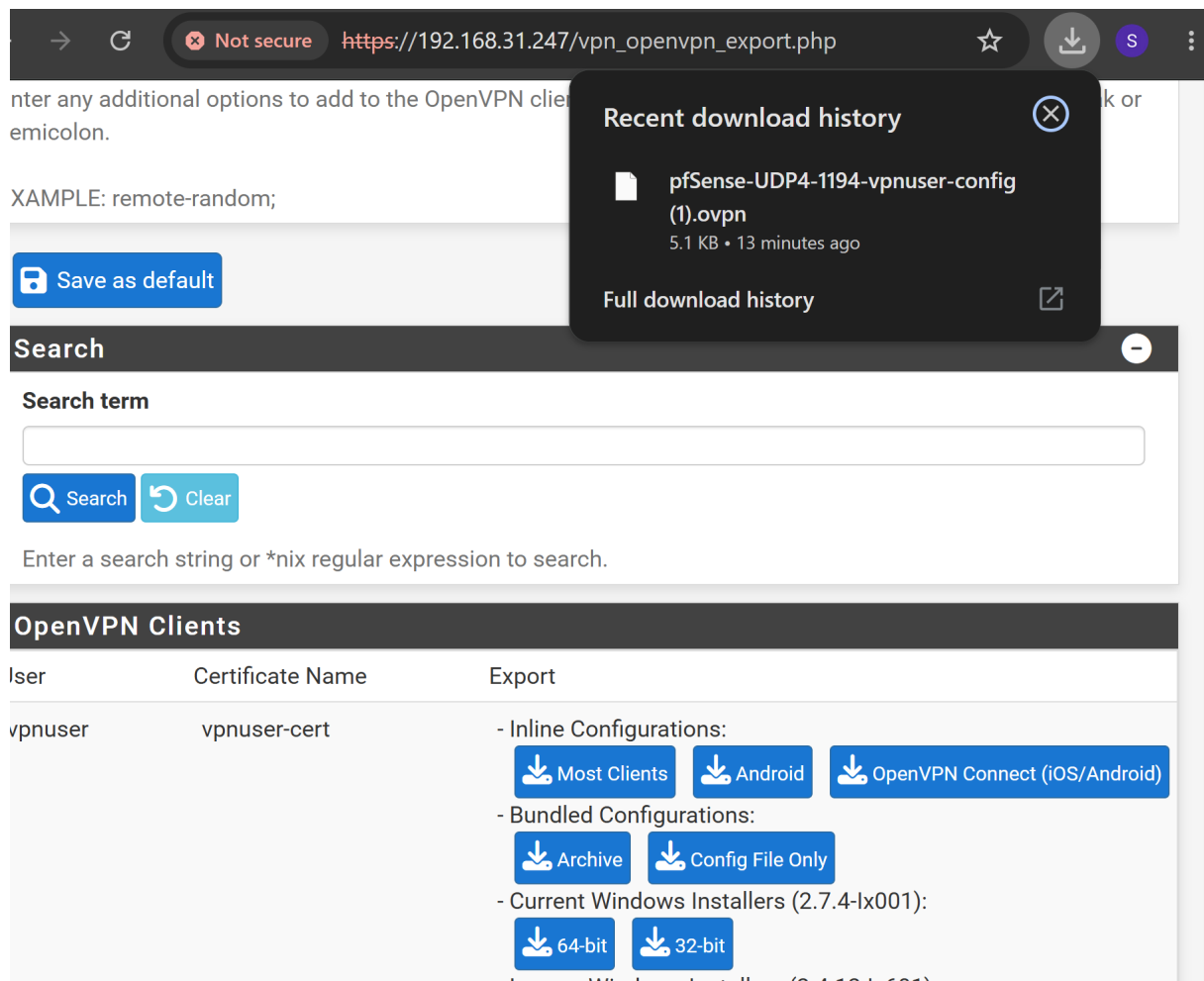


FIGURE 23 – Fichier de configuration .ovpn exporté pour vpnuser

Le fichier `pfSense-UDP4-1194-vpnuser-config.ovpn` est téléchargé et copié dans le répertoire `C:\Users\W11B\OpenVPN\config\` de la machine Windows. Le client OpenVPN GUI version 2.7.4 est installé sur Windows 11.

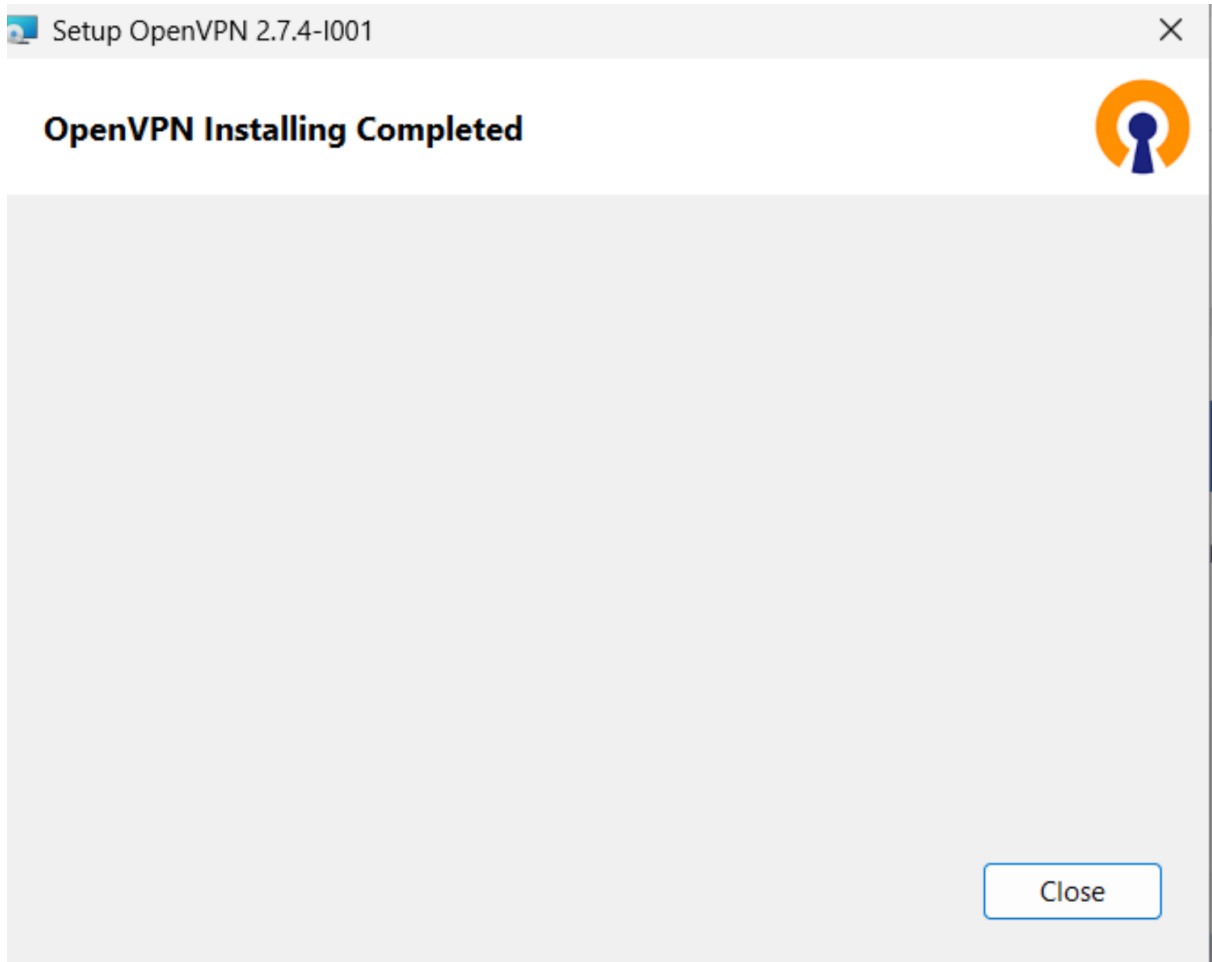


FIGURE 24 – Installation du client OpenVPN GUI sur Windows 11

3.7 Étape 7 Connexion et tests VPN

Après le lancement du client OpenVPN GUI en tant qu'administrateur, la connexion est initiée avec les identifiants de l'utilisateur `vpnuser`. Le tunnel VPN est établi avec succès et une adresse IP `10.8.0.2` est assignée au client Windows dans le réseau tunnel `10.8.0.0/24`.

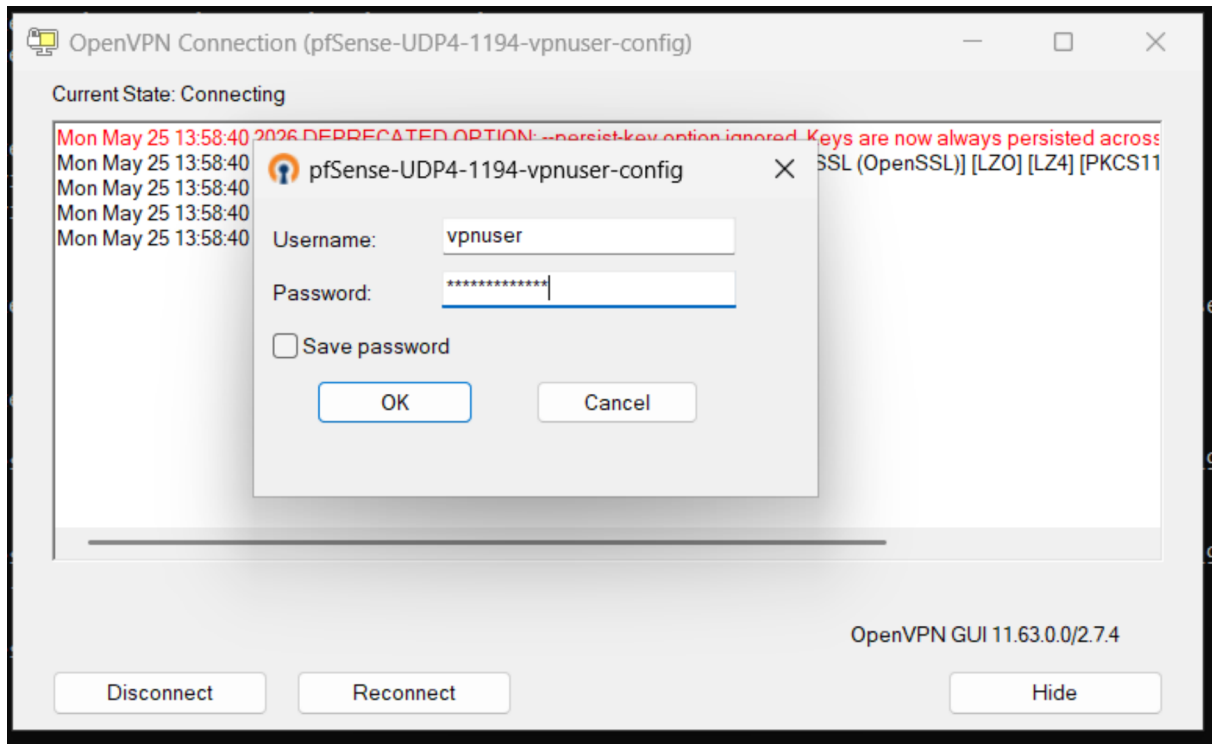


FIGURE 25 – Authentification de l'utilisateur vpnuser

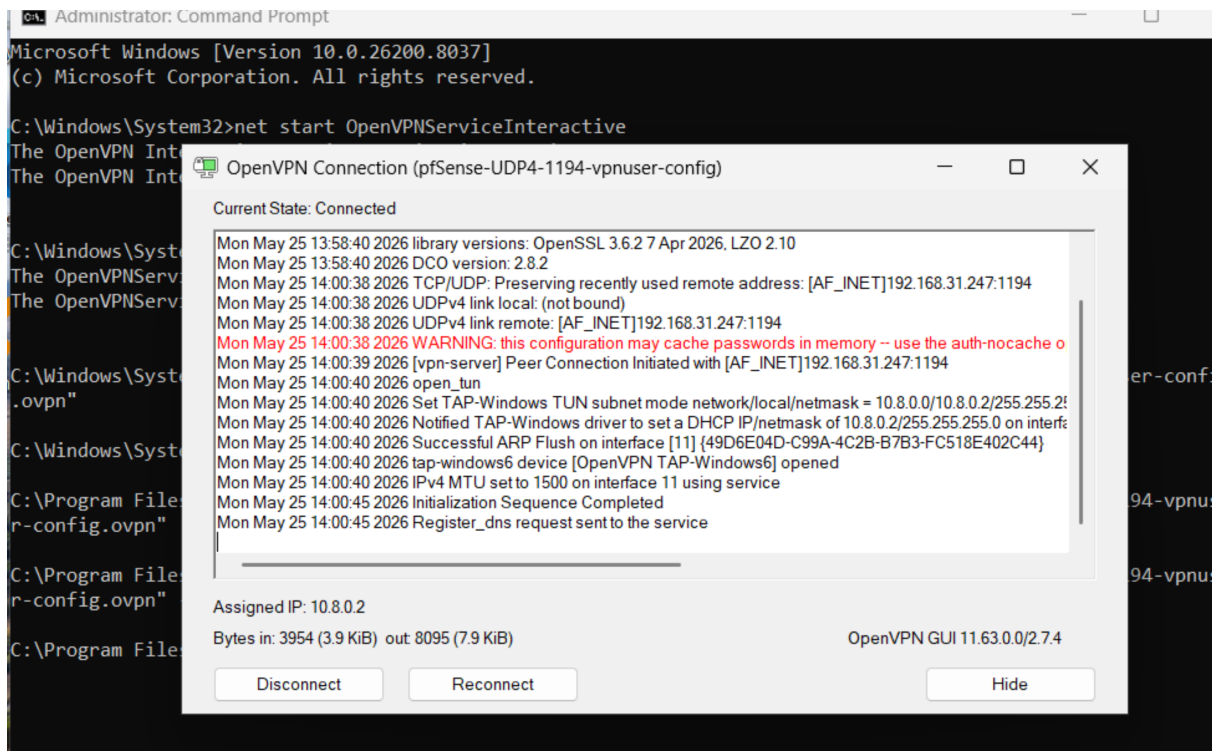


FIGURE 26 – Tunnel VPN établi avec succès état : Connected

Les tests de connectivité suivants ont été réalisés depuis Windows 11 après l'établissement du tunnel VPN :

- **Adresse IP VPN** : 10.8.0.2 attribuée automatiquement par le serveur OpenVPN.
- **Ping vers pfSense LAN** : 192.168.56.1 succès, confirmant la connectivité avec la passerelle interne.
- **Ping vers Ubuntu Server** : 192.168.56.10 succès, confirmant l'accès au serveur interne via le tunnel.
- **Accès SSH** : connexion SSH établie avec succès vers Ubuntu Server depuis Windows via le VPN.
- **Accès HTTP** : la page par défaut d'Apache2 est accessible depuis le navigateur Windows via le tunnel VPN.

```
C:\Windows\System32>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::7e7f:655f:37be:9840%10
    IPv4 Address. . . . . : 192.168.31.165
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.31.1

Unknown adapter OpenVPN TAP-Windows6:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::8336:7997:be75:f7a1%11
    IPv4 Address. . . . . : 10.8.0.2
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Unknown adapter OpenVPN Data Channel Offload:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

C:\Windows\System32>
```

FIGURE 27 – Adresse IP VPN 10.8.0.2 attribuée à Windows

```
C:\Windows\System32>ping 192.168.56.1

Pinging 192.168.56.1 with 32 bytes of data:
Reply from 192.168.56.1: bytes=32 time=4ms TTL=64
Reply from 192.168.56.1: bytes=32 time=3ms TTL=64
Reply from 192.168.56.1: bytes=32 time=3ms TTL=64

Ping statistics for 192.168.56.1:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 3ms, Maximum = 4ms, Average = 3ms
Control-C
^C
C:\Windows\System32>
```

FIGURE 28 – Ping réussi vers 192.168.56.1 pfSense LAN via VPN

```
C:\Windows\System32>ping 192.168.56.10

Pinging 192.168.56.10 with 32 bytes of data:
Reply from 192.168.56.10: bytes=32 time=8ms TTL=63
Reply from 192.168.56.10: bytes=32 time=4ms TTL=63
Reply from 192.168.56.10: bytes=32 time=4ms TTL=63
Reply from 192.168.56.10: bytes=32 time=249ms TTL=63

Ping statistics for 192.168.56.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 4ms, Maximum = 249ms, Average = 66ms
C:\Windows\System32>
```

FIGURE 29 – Ping réussi vers 192.168.56.10 Ubuntu Server via VPN

3.8 Étape 8 Activation des services sur Ubuntu Server

Deux services essentiels sont installés et activés sur Ubuntu Server : Apache2 pour l'hébergement web et OpenSSH pour l'administration distante sécurisée.

```
salma@salma:~$ sudo systemctl start ssh
salma@salma:~$ sudo systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-05-25 17:04:31 UTC; 1min 31s ago
   TriggeredBy: • ssh.socket
   Docs: man:sshd(8)
         man:sshd_config(5)
   Process: 1118 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1120 (sshd)
     Tasks: 1 (limit: 4601)
    Memory: 2.1M (peak: 2.4M)
       CPU: 38ms
    CGroup: /system.slice/ssh.service
            └─1120 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 25 17:04:31 salma systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 25 17:04:31 salma sshd[1120]: Server listening on 0.0.0.0 port 22.
May 25 17:04:31 salma sshd[1120]: Server listening on :: port 22.
May 25 17:04:31 salma systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
salma@salma:~$ sudo systemctl apache2 ssh
Unknown command verb 'apache2', did you mean 'cancel'?
salma@salma:~$ sudo systemctl status apache2
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-25 16:57:19 UTC; 9min ago
   Docs: https://httpd.apache.org/docs/2.4/
   Process: 773 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 827 (apache2)
     Tasks: 55 (limit: 4601)
    Memory: 7.7M (peak: 7.9M)
       CPU: 455ms
    CGroup: /system.slice/apache2.service
            └─827 /usr/sbin/apache2 -k start
              └─830 /usr/sbin/apache2 -k start
                └─831 /usr/sbin/apache2 -k start

May 25 16:57:16 salma systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 25 16:57:18 salma apachectl[806]: AH00558: apache2: Could not reliably determine the
May 25 16:57:19 salma systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-17/17 (END)
```

FIGURE 30 – Activation des services Apache2 et SSH sur Ubuntu Server

3.8.1 Test d'accès SSH

La connexion SSH est testée depuis Windows via le tunnel VPN avec la commande `ssh salma@192.168.56.10`. L'accès est établi avec succès, confirmant que le service SSH est opérationnel et accessible uniquement via le VPN.

```

salma@salma: ~
salma@192.168.56.10's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-117-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May 25 05:13:02 PM UTC 2026

System load:  0.08          Processes:           110
Usage of /:   44.4% of 11.21GB Users logged in:       1
Memory usage: 6%           IPv4 address for enp0s3: 192.168.56.10
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

salma@salma:~$ |
    
```

FIGURE 31 – Connexion SSH réussie vers Ubuntu Server via VPN

3.8.2 Test d'accès HTTP

Le serveur web Apache2 est testé en accédant à l'adresse `http://192.168.56.10` depuis le navigateur de Windows 11. La page par défaut d'Apache2 s'affiche correctement, confirmant que le service web est opérationnel et accessible via le tunnel VPN.



FIGURE 32 – Accès réussi à la page Apache2 depuis Windows via VPN

3.9 Conclusion de la Phase 2

La phase de déploiement OpenVPN est complète. L'infrastructure VPN Client-to-Site est pleinement opérationnelle :

- Le tunnel VPN chiffré en AES-256-CBC est établi avec succès entre Windows 11 et pfSense.
- L'authentification double facteur (certificat + mot de passe) est fonctionnelle.
- Le client distant obtient une adresse IP dans le réseau 10.8.0.0/24 et accède au réseau interne 192.168.56.0/24.
- Les services SSH et HTTP d'Ubuntu Server sont accessibles exclusivement via le tunnel VPN sécurisé.

4 Partie 3 Durcissement du serveur Ubuntu

L'objectif de cette phase est de réduire la surface d'attaque du serveur Ubuntu en appliquant un ensemble de mesures de sécurité au niveau du système, du réseau, de l'authentification et des accès distants. Cette phase suit les bonnes pratiques de sécurisation des systèmes Linux en environnement professionnel.

4.1 Étape 1 Mise à jour du système

La première action consiste à mettre à jour l'ensemble des paquets installés sur le système. Cette étape est fondamentale car elle permet de corriger les vulnérabilités connues et d'appliquer les derniers correctifs de sécurité publiés par les mainteneurs d'Ubuntu. La commande `apt update && apt upgrade` synchronise la liste des paquets disponibles puis installe les nouvelles versions.

```

salma@salma:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for salma:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://ma.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://ma.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://ma.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,191 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [230 kB]
Get:7 http://ma.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,692 kB]
Get:8 http://ma.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [329 kB]
Fetched 3,693 kB in 5s (811 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
salma@salma:~$ sudo apt autoremove -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
salma@salma:~$
    
```

FIGURE 33 – Mise à jour du système

4.2 Étape 2 Suppression des services inutiles

Chaque service actif sur un serveur représente une surface d'attaque potentielle. Le principe du moindre privilège impose de n'exécuter que les services strictement nécessaires. La commande `systemctl list-units -type=service -state=running` permet de lister tous les services actifs afin d'identifier ceux qui sont inutiles.

```

salma@salma:~$ systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
apache2.service                    loaded active running The Apache HTTP Server
cron.service                        loaded active running Regular background program processing daemon
dbus.service                        loaded active running D-Bus System Message Bus
fwupd.service                       loaded active running Firmware update daemon
getty@tty1.service                 loaded active running Getty on tty1
ModemManager.service               loaded active running Modem Manager
multipathd.service                 loaded active running Device-Mapper Multipath Device Controller
packagekit.service                loaded active running PackageKit Daemon
polkit.service                     loaded active running Authorization Manager
rsyslog.service                    loaded active running System Logging Service
ssh.service                         loaded active running OpenBSD Secure Shell server
systemd-journald.service            loaded active running Journal Service
systemd-logind.service              loaded active running User Login Management
systemd-networkd.service            loaded active running Network Configuration
systemd-resolved.service            loaded active running Network Name Resolution
systemd-timesyncd.service            loaded active running Network Time Synchronization
systemd-udevd.service               loaded active running Rule-based Manager for Device Events and Files
udisks2.service                    loaded active running Disk Manager
unattended-upgrades.service          loaded active running Unattended Upgrades Shutdown
upower.service                      loaded active running Daemon for power management
user@1000.service                   loaded active running User Manager for UID 1000

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
          ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
          SUB     → The low-level unit activation state, values depend on unit type.

21 loaded units listed.
salma@salma:~$
    
```

FIGURE 34 – Services en cours d'exécution dans le système

L'analyse révèle plusieurs services inutiles dans notre contexte : **ModemManager** (gestionnaire de modems, sans utilité sur un serveur virtuel), **multipathd** (gestion des chemins disques redondants, inutile en VM) et **fwupd** (mise à jour du firmware matériel, sans pertinence sur une machine virtuelle). Ces services sont désactivés via **systemctl disable -now**, réduisant le nombre de services actifs de 21 à 18.

```

salma@salma:~$ sudo systemctl disable --now fwupd
The unit files have no installation config (WantedBy=, RequiredBy=, UpheldBy=,
Also=, or Alias= settings in the [Install] section, and DefaultInstance= for
template units). This means they are not meant to be enabled or disabled using systemctl.

Possible reasons for having these kinds of units are:
• A unit may be statically enabled by being symlinked from another unit's
  .wants/, .requires/, or .upholds/ directory.
• A unit's purpose may be to act as a helper for some other unit which has
  a requirement dependency on it.
• A unit may be started when needed via activation (socket, path, timer,
  D-Bus, udev, scripted systemctl call, ...).
• In case of template units, the unit is meant to be enabled with some
  instance name specified.
salma@salma:~$ sudo systemctl disable --now ModemManager
Failed to disable unit: Unit file ModemManager.service does not exist.
salma@salma:~$ sudo systemctl disable --now ModemManager
Removed "/etc/systemd/system/dbus-org.freedesktop.ModemManager1.service".
Removed "/etc/systemd/system/multi-user.target.wants/ModemManager.service".
salma@salma:~$ sudo systemctl disable --now multipathd
Removed "/etc/systemd/system/sysinit.target.wants/multipathd.service".
Removed "/etc/systemd/system/sockets.target.wants/multipathd.socket".
Disabling 'multipathd.service', but its triggering units are still active:
multipathd.socket
Stopping 'multipathd.service', but its triggering units are still active:
multipathd.socket
salma@salma:~$ sudo systemctl disable --now packagekit
The unit files have no installation config (WantedBy=, RequiredBy=, UpheldBy=,
Also=, or Alias= settings in the [Install] section, and DefaultInstance= for
template units). This means they are not meant to be enabled or disabled using systemctl.

Possible reasons for having these kinds of units are:
• A unit may be statically enabled by being symlinked from another unit's
  .wants/, .requires/, or .upholds/ directory.
• A unit's purpose may be to act as a helper for some other unit which has
  a requirement dependency on it.
• A unit may be started when needed via activation (socket, path, timer,
  D-Bus, udev, scripted systemctl call, ...).
• In case of template units, the unit is meant to be enabled with some
  instance name specified.
salma@salma:~$ _
    
```

FIGURE 35 – Désactivation des services inutiles du système

4.3 Étape 3 Désactivation d'IPv6

IPv6 n'étant pas utilisé dans notre architecture réseau, son maintien actif représente une surface d'attaque inutile. La désactivation est réalisée de manière permanente via le fichier `/etc/sysctl.conf` en ajoutant les paramètres suivants :

- `net.ipv6.conf.all.disable_ipv6 = 1`
- `net.ipv6.conf.default.disable_ipv6 = 1`
- `net.ipv6.conf.lo.disable_ipv6 = 1`

```
#####
# Magic system request Key
# 0=disable, 1=enable all, >1 bitmask of sysrq functions
# See https://www.kernel.org/doc/html/latest/admin-guide/sysrq.html
# for what other values do
#kernel.sysrq=438
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1_

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify
```

FIGURE 36 – Ajout des paramètres de désactivation IPv6 dans /etc/sysctl.conf

La commande `sysctl -p` applique immédiatement ces paramètres sans redémarrage. La valeur 1 retournée confirme la désactivation complète d'IPv6 sur toutes les interfaces.

```
salma@salma:~$ sudo sysctl -p
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1_
salma@salma:~$ _
```

FIGURE 37 – IPv6 désactivé avec succès

4.4 Étape 4 Durcissement du noyau via sysctl

Le fichier `/etc/sysctl.conf` permet de configurer des paramètres de sécurité au niveau du noyau Linux. Les protections suivantes ont été ajoutées :

- **Protection anti-spoofing** (`rp_filter = 1`) : active la vérification du chemin inverse des paquets, bloquant les paquets dont l'adresse source est falsifiée.
- **Désactivation des redirections ICMP** (`accept_redirects = 0`) : empêche un attaquant de modifier la table de routage via des messages ICMP redirect malveillants.
- **Désactivation du source routing** (`accept_source_route = 0`) : interdit les paquets dont l'expéditeur impose le chemin de routage.
- **Protection SYN flood** (`tcp_syncookies = 1`) : active les SYN cookies pour résister aux attaques par saturation de la file d'attente TCP.
- **Ignore broadcasts ICMP** (`icmp_echo_ignore_broadcasts = 1`) : empêche le serveur de répondre aux pings broadcast, utilisés dans les attaques Smurf.
- **Journalisation des paquets suspects** (`log_martians = 1`) : enregistre les paquets avec des adresses sources impossibles ou suspectes.

```
#protection anti-spoofing
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1

#desactiver redirections ICMP
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0

#desactiver source routing
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0

#protection STN flood
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2

#ignorer broadcast ICMP
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_ignore_bogus_error_responses = 1

#logger packets suspects
net.ipv4.conf.all.log_martians = 1_
```

FIGURE 38 – Ajout des paramètres de sécurité réseau dans `/etc/sysctl.conf`

Tous les paramètres sont appliqués immédiatement via `sysctl -p` sans aucune erreur.

```

salma@salma:~$ sudo sysctl -p
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_ignore_bogus_error_responses = 1
net.ipv4.conf.all.log_martians = 1
salma@salma:~$ _
    
```

FIGURE 39 – Tous les paramètres sysctl appliqués avec succès

4.5 Étape 5 Configuration du pare-feu UFW

UFW (*Uncomplicated Firewall*) est l'interface simplifiée d'iptables sur Ubuntu. La politique appliquée repose sur le principe du *deny by default* : tout trafic entrant est bloqué par défaut, et seuls les services nécessaires sont autorisés depuis des sources de confiance.

Les règles configurées autorisent :

- **SSH (port 22)** uniquement depuis le réseau VPN 10.8.0.0/24 et le réseau LAN 192.168.56.0/24.
- **HTTP (port 80)** uniquement depuis ces mêmes réseaux de confiance.
- Tout autre trafic entrant est **rejeté**.

```

salma@salma:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
salma@salma:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
salma@salma:~$ sudo ufw allow from 10.8.0.0/24 to any port 22 proto tcp comment "SSH via VPN"
Rule added
salma@salma:~$ sudo ufw allow from 10.8.0.0/24 to any port 80 proto tcp comment "HTTP via VPN"
Rule added
salma@salma:~$ sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment "SSH via LAN"
Rule added
salma@salma:~$ sudo ufw allow from 192.168.56.0/24 to any port 80 proto tcp comment "HTTP via LAN"
Rule added
salma@salma:~$ sudo ufw enable
Firewall is active and enabled on system startup
salma@salma:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
Anywhere ALLOW IN 192.168.56.0/24
Anywhere ALLOW IN 10.8.0.0/24
Anywhere on enp0s3 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
22/tcp ALLOW IN 10.8.0.0/24 # SSH via VPN
80/tcp ALLOW IN 10.8.0.0/24 # HTTP via VPN
22/tcp ALLOW IN 192.168.56.0/24 # SSH via LAN
80/tcp ALLOW IN 192.168.56.0/24 # HTTP via LAN
Anywhere (v6) on enp0s3 ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
salma@salma:~$
    
```

FIGURE 40 – Configuration initiale du pare-feu UFW

Après vérification, les règles trop permissives autorisant le trafic depuis *Anywhere* (règles 1, 2, 3, 4, 5, 10, 11, 12) sont supprimées. Seules les 4 règles strictement nécessaires sont conservées, garantissant que les services ne sont accessibles que depuis le tunnel VPN ou le réseau interne.

```

Deleting:
  allow 80/tcp
Proceed with operation (y|n)? y
Rule deleted (v6)
salma@salma:~$ sudo ufw delete 11
Deleting:
  allow 22/tcp
Proceed with operation (y|n)? y
Rule deleted (v6)
salma@salma:~$ sudo ufw delete 10
Deleting:
  allow in on enp0s3
Proceed with operation (y|n)? y
Rule deleted (v6)
salma@salma:~$ sudo ufw delete 5
Deleting:
  allow 80/tcp
Proceed with operation (y|n)? y
Rule deleted
salma@salma:~$ sudo ufw delete 4
Deleting:
  allow 22/tcp
Proceed with operation (y|n)? y
Rule deleted
salma@salma:~$ sudo ufw delete 3
Deleting:
  allow in on enp0s3
Proceed with operation (y|n)? y
Rule deleted
salma@salma:~$ sudo ufw delete 2
Deleting:
  allow from 10.8.0.0/24
Proceed with operation (y|n)? y
Rule deleted
salma@salma:~$ sudo ufw delete 1
Deleting:
  allow from 192.168.56.0/24
Proceed with operation (y|n)? y
Rule deleted
salma@salma:~$ sudo ufw status numbered
Status: active

    To      Action      From
    --      -
[ 1] 22/tcp  ALLOW IN    10.8.0.0/24    # SSH via VPN
[ 2] 80/tcp  ALLOW IN    10.8.0.0/24    # HTTP via VPN
[ 3] 22/tcp  ALLOW IN    192.168.56.0/24 # SSH via LAN
[ 4] 80/tcp  ALLOW IN    192.168.56.0/24 # HTTP via LAN
salma@salma:~$
    
```

FIGURE 41 – UFW parfaitement configuré avec les 4 règles strictes uniquement

4.6 Étape 6 Verrouillage du compte root

Le compte `root` est la cible principale des attaques par force brute. La commande `passwd -l root` verrouille le compte en rendant toute authentification directe impossible. Le statut `L` (Locked) confirmé par `passwd -S root` valide cette protection.

```

salma@salma:~$ sudo passwd -l root
passwd: password changed.
salma@salma:~$ sudo passwd -S root
root L 2026-02-10 0 99999 7 -1
salma@salma:~$
    
```

FIGURE 42 – Compte root verrouillé statut L confirmé

4.7 Étape 7 Protection du chargeur de démarrage GRUB

Sans protection, un attaquant ayant accès à la console peut modifier les paramètres de démarrage pour accéder au système en mode recovery sans authentification, contournant toutes les mesures de sécurité. Un mot de passe GRUB est généré via `grub-mkpasswd-pbkdf2` qui produit un hash sécurisé au format PBKDF2-SHA512.

```

salma@salma:~$ sudo grub-mkpasswd-pbkdf2
Enter password:
Reenter password:
PBKDF2 hash of your password is grub.pbkdf2.sha512.10000.CC8B3841228DF3303BB1D5FFD1E97EBA9DA7F1FB2C9CF9F6B05046F3B772F813FF8A3C8E866BC683432CD8C6E6F5C1B86C18F5E
9963A76DF782EA9E3407D63F1.13C99544062D1006F901B0892F4038322FD84B9F73AB40E63D4BA44245CE23E68737F5D54DAE0437C883A9B77F78022426826F38EBE93CD57C8014DFF9F479E3D
salma@salma:~$ _
    
```

FIGURE 43 – Génération du hash GRUB avec `grub-mkpasswd-pbkdf2`

Ce hash est ensuite ajouté au fichier `/etc/grub.d/40_custom` avec les directives `set superusers` et `password_pbkdf2`, définissant `root` comme seul superutilisateur autorisé à modifier les options de démarrage.

```

GNU nano 7.2 /etc/grub.d/40_custom *
#!/bin/sh
exec tail -n +3 $0
# This file provides an easy way to add custom menu entries. Simply type the
# menu entries you want to add after this comment. Be careful not to change
# the 'exec tail' line above.
set superusers="root"
password_pbkdf2 rootgrub.pbkdf2.sha512.10000.CC8B3841228DF3303BB1D5FFD1E97EBA9DA7F1FB2C9CF9F6B05046F3B772F813FF8A3C8E866BC683432CD8C6E6F5
    
```

FIGURE 44 – Ajout du hash de protection dans `/etc/grub.d/40_custom`

La commande `update-grub` régénère la configuration de démarrage en intégrant la protection par mot de passe.

```

salma@salma:~$ sudo update-grub
[sudo] password for salma:
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
salma@salma:~$ _
    
```

FIGURE 45 – GRUB mis à jour avec succès

5 Sécurisation PAM

PAM (*Pluggable Authentication Modules*) est le système d'authentification modulaire de Linux. Il permet de définir des politiques de mots de passe et de contrôle d'accès appliquées à l'ensemble du système.

5.1 Expiration des mots de passe

La politique d'expiration est configurée dans `/etc/login.defs` puis appliquée à l'utilisateur via la commande `chage` :

- **PASS_MAX_DAYS 90** : le mot de passe expire après 90 jours, forçant un renouvellement régulier.
- **PASS_MIN_DAYS 7** : impossible de changer le mot de passe avant 7 jours, évitant qu'un utilisateur revienne immédiatement à son ancien mot de passe.
- **PASS_WARN_AGE 14** : l'utilisateur est averti 14 jours avant l'expiration de son mot de passe.

```

# Password aging controls:
#
#     PASS_MAX_DAYS   Maximum number of days a password may be used.
#     PASS_MIN_DAYS   Minimum number of days allowed between password changes.
#     PASS_WARN_AGE   Number of days warning given before a password expires.
#
PASS_MAX_DAYS   90
PASS_MIN_DAYS    7
PASS_WARN_AGE   14
#
# Min/max values for automatic uid selection in useradd
#
    
```

FIGURE 46 – Politique d'expiration des mots de passe configurée avec succès

5.2 Complexité des mots de passe

Le module `pam_pwquality` impose des règles de complexité aux nouveaux mots de passe via le fichier `/etc/security/pwquality.conf` :

- **minlen = 12** : longueur minimale de 12 caractères.
- **dcredit = -1** : au moins 1 chiffre obligatoire.
- **ucredit = -1** : au moins 1 majuscule obligatoire.
- **lcredit = -1** : au moins 1 minuscule obligatoire.
- **ocredit = -1** : au moins 1 caractère spécial obligatoire.
- **maxrepeat = 3** : interdit plus de 3 caractères identiques consécutifs.

```

# /etc/security/pwquality.conf
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
minclass = 3
maxrepeat = 3

salma@salma:~$ sudo grep -v "^#" /etc/security/pwquality.conf | grep -v "^$"
minlen = 12
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
minclass = 3
maxrepeat = 3
salma@salma:~$ _
    
```

FIGURE 47 – Règles de complexité des mots de passe configurées avec succès

5.3 Verrouillage après tentatives échouées

Le module `pam_faillock` verrouille automatiquement un compte après plusieurs échecs d'authentification. Il est configuré dans `/etc/pam.d/common-auth` et `/etc/pam.d/common-account`. La politique appliquée bloque le compte après **5 tentatives échouées** et le déverrouille automatiquement après **15 minutes** (900 secondes), protégeant efficacement contre les attaques par force brute sur les comptes locaux.

```

salma@salma:~$ sudo grep faillock /etc/pam.d/common-auth
auth required pam_faillock.so preauth silent deny=5 unlock_time=900
salma@salma:~$ sudo grep faillock /etc/pam.d/common-account
account required pam_faillock.so
salma@salma:~$
    
```

FIGURE 48 – Module `pam_faillock` configuré avec succès

6 Sécurisation SSH

SSH est le principal vecteur d'administration distante du serveur. Sa sécurisation est donc critique. Les paramètres suivants ont été configurés dans `/etc/ssh/sshd_config` :

- **PermitRootLogin no** : interdit la connexion SSH directe avec le compte root. L'administrateur doit utiliser un compte standard puis élever ses privilèges via `sudo`.
- **MaxAuthTries 3** : limite à 3 le nombre de tentatives d'authentification par connexion, ralentissant les attaques par force brute.
- **LoginGraceTime 30** : la connexion est fermée automatiquement si l'authentification n'est pas complétée en 30 secondes.
- **X11Forwarding no** : désactive le transfert graphique X11, inutile sur un serveur et source de vulnérabilités potentielles.
- **PermitEmptyPasswords no** : interdit les connexions avec un mot de passe vide.
- **ClientAliveInterval 300** : envoie un signal toutes les 5 minutes pour détecter les sessions inactives.
- **ClientAliveCountMax 2** : ferme automatiquement la session après 2 signaux sans réponse, soit 10 minutes d'inactivité totale.

```

PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem sftp /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
PermitEmptyPasswords no
ClientAliveInterval 300
ClientAliveCountMax 2
PermitRootLogin no

salma@salma:~$ sudo systemctl restart ssh
salma@salma:~$ sudo sshd -T | grep -E "permitrootlogin|maxauthtries|x11forwarding|peremptypasswords"
maxauthtries 3
x11forwarding no
peremptypasswords no
salma@salma:~$ sudo sshd -T | grep -E "permitrootlogin|maxauthtries|x11forwarding|peremptypasswords"
maxauthtries 3
permitrootlogin no
x11forwarding no
peremptypasswords no
salma@salma:~$
    
```

FIGURE 49 – SSH parfaitement sécurisé vérification des paramètres appliqués

Conclusion de la Phase 3

L'ensemble des mesures de durcissement appliquées transforme le serveur Ubuntu en un système renforcé conforme aux bonnes pratiques de sécurité. La combinaison de la restriction réseau via UFW, du durcissement du noyau via sysctl, de la politique d'authentification PAM et de la sécurisation SSH réduit considérablement la surface d'attaque et renforce la résilience du serveur face aux menaces externes.

7 Test de la protection GRUB sur Ubuntu

La protection GRUB mise en place lors de la phase de durcissement est validée au démarrage du serveur. Lorsque la machine Ubuntu est démarrée, le menu GRUB s'affiche normalement avec les options de démarrage disponibles.

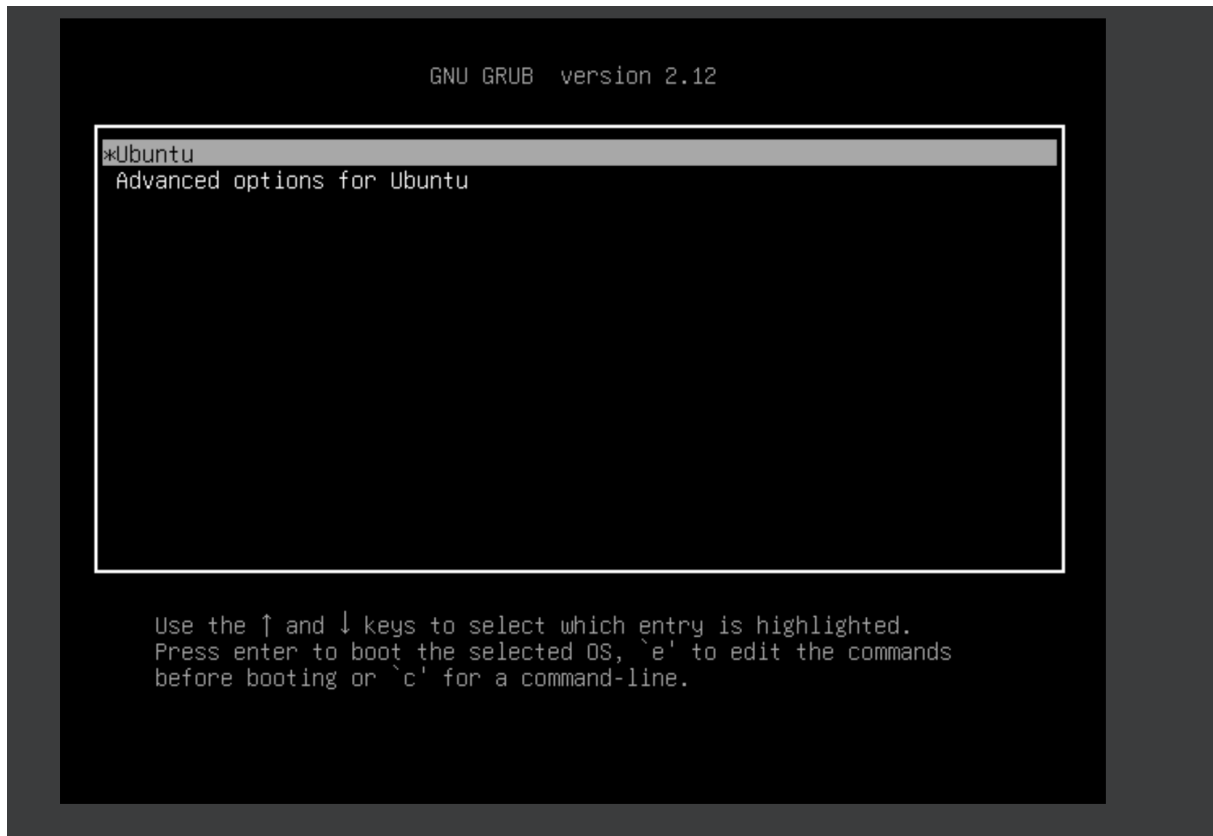


FIGURE 50 – Menu GRUB la protection est bien active sur le serveur

Dès qu'une tentative de modification des options de démarrage est effectuée (touche **e** pour éditer), GRUB demande une authentification avant d'autoriser tout accès. Cette mesure empêche un attaquant ayant accès physique ou console à la machine de modifier les paramètres du noyau ou de démarrer en mode recovery sans connaître les identifiants définis lors de la configuration.

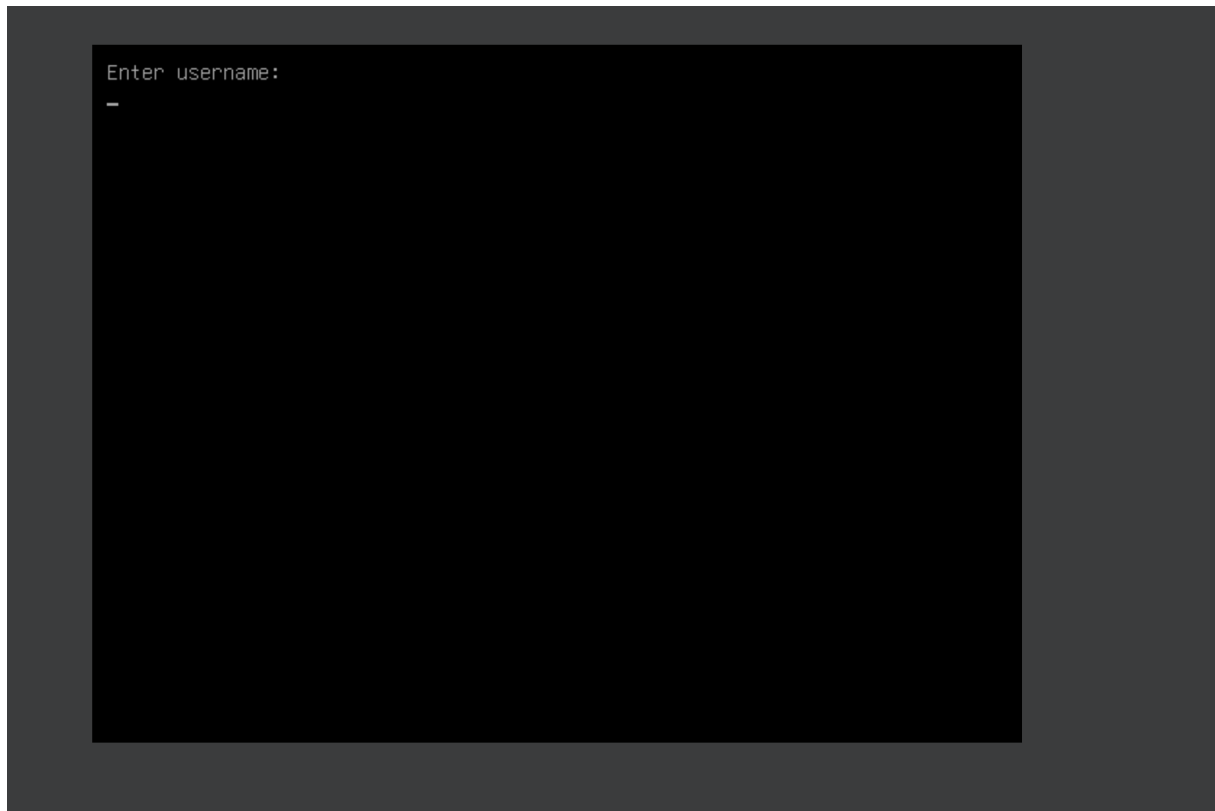


FIGURE 51 – GRUB demande les identifiants de sécurité avant toute modification

Après saisie des identifiants corrects du superutilisateur **root** défini lors de la configuration GRUB, l'accès est accordé et le serveur démarre normalement. Cette validation confirme que la protection GRUB est pleinement opérationnelle et que le serveur est protégé contre toute manipulation non autorisée du chargeur de démarrage.

```
Ubuntu 24.04.4 LTS salma tty1
salma login: salma
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-117-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Tue May 26 01:22:49 PM UTC 2026

System load:  0.02               Processes:            100
Usage of /:   44.5% of 11.21GB   Users logged in:     0
Memory usage: 6%                IPv4 address for enp0s3: 192.168.56.10
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

salma@salma:~$
```

FIGURE 52 – Démarrage réussi du serveur Ubuntu après authentification GRUB

8 Partie 4 Services sur Ubuntu

Cette phase valide le bon fonctionnement des services hébergés sur le serveur Ubuntu et leur accessibilité depuis le client Windows via le tunnel VPN sécurisé. Les deux services principaux déployés sont Apache2 pour l'hébergement web et OpenSSH pour l'administration distante sécurisée.

8.1 Vérification des services

Les deux services sont actifs et configurés pour démarrer automatiquement au lancement du système. Le service Apache2 écoute sur le port 80 (HTTP) et le service SSH écoute sur le port 22, tous deux accessibles uniquement depuis les réseaux de confiance définis dans les règles UFW.

8.2 Test de connectivité Ping depuis Windows

Depuis le client Windows 11 connecté au tunnel VPN, un test de ping vers l'adresse 192.168.56.10 confirme la connectivité réseau de bout en bout entre le client distant et le serveur interne via le tunnel chiffré. Les paquets sont reçus avec 0% de perte, validant le bon fonctionnement du tunnel VPN et des règles ICMP configurées dans UFW.

```

salma@salma:~$ sudo systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
  Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-05-26 13:45:16 UTC; 5s ago
  TriggeredBy: • ssh.socket
    Docs: man:sshd(8)
           man:sshd_config(5)
  Process: 2164 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 2165 (sshd)
   Tasks: 1 (limit: 4601)
  Memory: 2.1M (peak: 2.4M)
    CPU: 53ms
  CGroup: /system.slice/ssh.service
          └─2165 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 26 13:45:16 salma systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 26 13:45:16 salma sshd[2165]: Server listening on 0.0.0.0 port 22.
May 26 13:45:16 salma sshd[2165]: Server listening on :: port 22.
May 26 13:45:16 salma systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
salma@salma:~$ sudo systemctl status apache2
• apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-05-26 13:17:43 UTC; 27min ago
    Docs: https://httpd.apache.org/docs/2.4/
 Main PID: 838 (apache2)
   Tasks: 55 (limit: 4601)
  Memory: 7.9M (peak: 8.1M)
    CPU: 452ms
  CGroup: /system.slice/apache2.service
          └─838 /usr/sbin/apache2 -k start
            └─839 /usr/sbin/apache2 -k start
              └─840 /usr/sbin/apache2 -k start

May 26 13:17:42 salma systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 26 13:17:43 salma apachectl[822]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name: []; please add the following line to your configuration file:
May 26 13:17:43 salma systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-16/16 (END)
    
```

FIGURE 53 – les deux services apache2 et shh sont active

```

salma@salma:~$ ss -tulnp | grep -E "80|22"
tcp    LISTEN 0      4096      0.0.0.0:22      0.0.0.0:*
tcp    LISTEN 0      511      *:80          *:*
tcp    LISTEN 0      4096      [::]:22      [::]:*
salma@salma:~$
    
```

FIGURE 54 – Les ports sont bien ouverts

```

C:\Users\W11B>ping 192.168.56.10

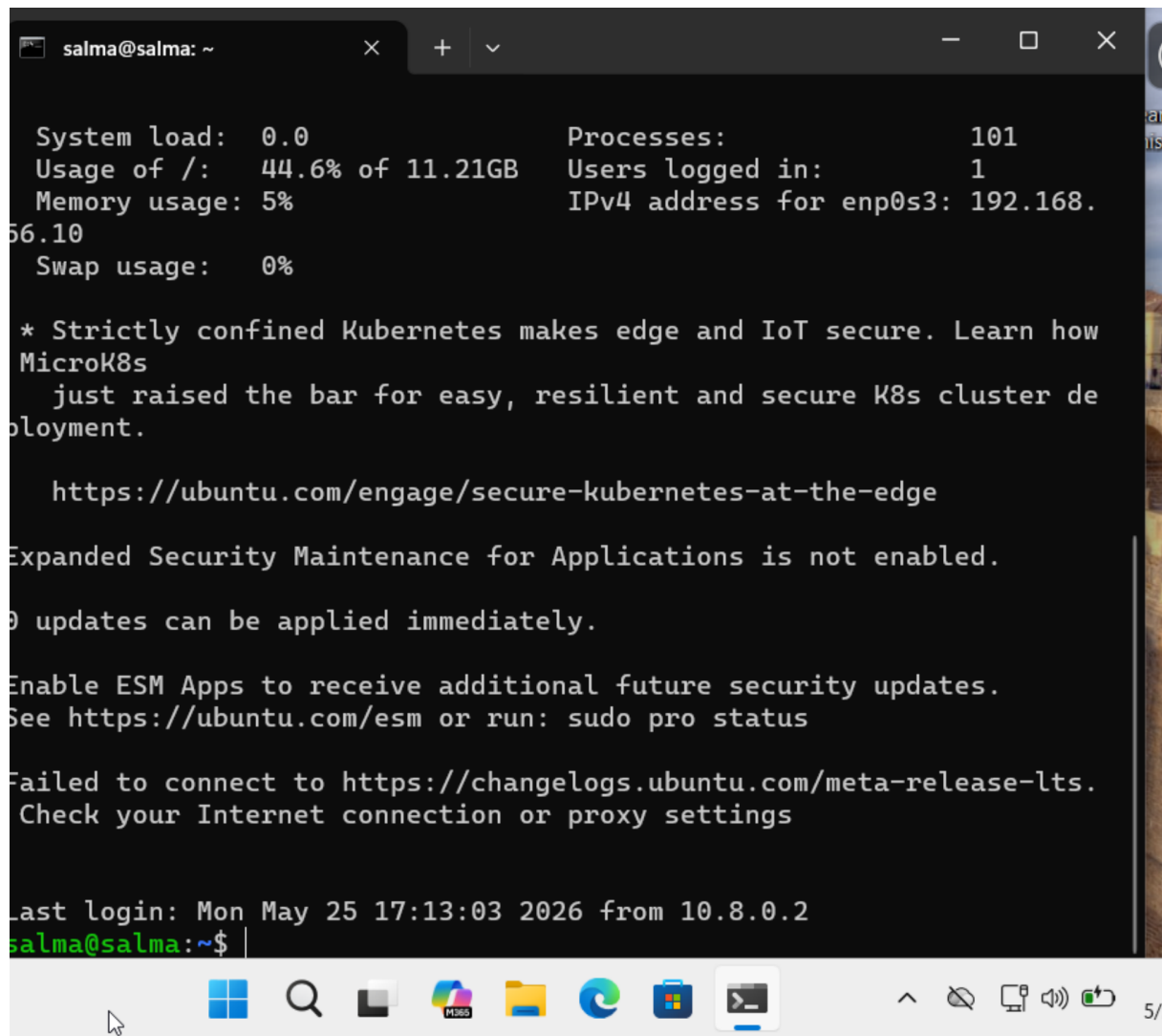
Pinging 192.168.56.10 with 32 bytes of data:
Reply from 192.168.56.10: bytes=32 time=9ms TTL=63
Reply from 192.168.56.10: bytes=32 time=4ms TTL=63

Ping statistics for 192.168.56.10:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 4ms, Maximum = 9ms, Average = 6ms
Control-C
^C
C:\Users\W11B>
    
```

8.3 Test d'accès SSH depuis Windows

La connexion SSH est établie depuis Windows 11 via le tunnel VPN avec la commande `ssh salma@192.168.56.10`. La connexion est établie avec succès, confirmant que :

- Le service SSH est opérationnel sur Ubuntu.
- Les règles UFW autorisent correctement le trafic SSH depuis le réseau VPN 10.8.0.0/24.
- Le tunnel VPN achemine correctement le trafic SSH chiffré entre Windows et Ubuntu.
- L'authentification par mot de passe fonctionne avec les politiques PAM configurées.



```

salma@salma: ~
System load: 0.0          Processes: 101
Usage of /: 44.6% of 11.21GB Users logged in: 1
Memory usage: 5%         IPv4 address for enp0s3: 192.168.56.10
Swap usage: 0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how
MicroK8s
just raised the bar for easy, resilient and secure K8s cluster de
ployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts.
Check your Internet connection or proxy settings

Last login: Mon May 25 17:13:03 2026 from 10.8.0.2
salma@salma:~$
    
```

FIGURE 56 – Connexion SSH réussie depuis Windows 11 vers Ubuntu Server via VPN

8.4 Test d'accès HTTP depuis Windows

Le serveur web Apache2 est testé en accédant à l'adresse `http://192.168.56.10` depuis le navigateur Edge de Windows 11. La page par défaut d'Apache2 *"It works!"* s'affiche correctement, confirmant que :

- Le service Apache2 est opérationnel et accessible via le tunnel VPN.

- Les règles UFW autorisent correctement le trafic HTTP (port 80) depuis le réseau VPN.
- Le serveur web répond correctement aux requêtes HTTP des clients distants connectés au VPN.

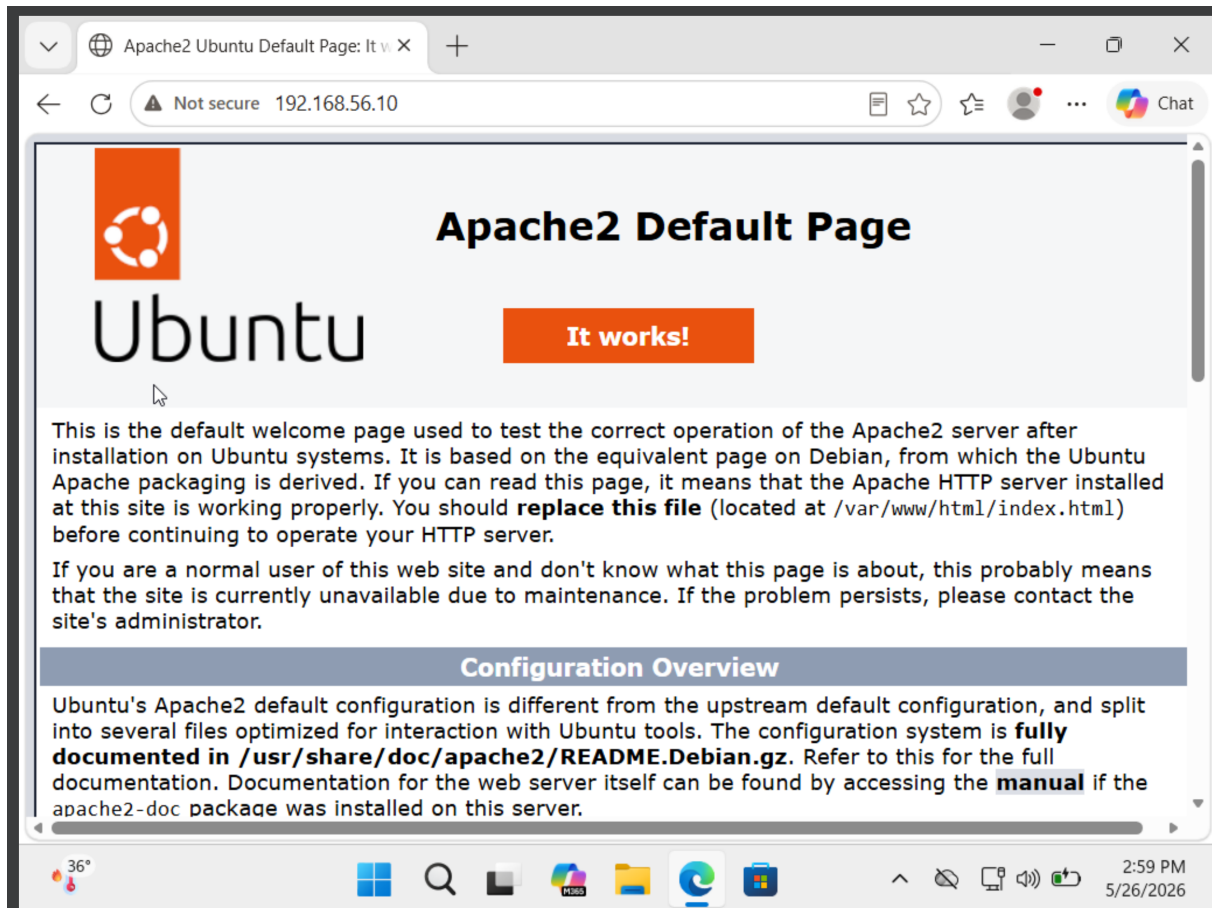


FIGURE 57 – Page Apache2 "It works!" accessible depuis Windows 11 via le tunnel VPN

Conclusion de la Phase 4

Les deux services essentiels du serveur Ubuntu sont pleinement opérationnels et accessibles exclusivement via le tunnel VPN sécurisé. Le serveur web Apache2 répond correctement aux requêtes HTTP et le service SSH permet l'administration distante sécurisée. L'ensemble de ces accès transite obligatoirement par le tunnel VPN chiffré en AES-256-CBC, garantissant la confidentialité et l'intégrité des communications.

9 Partie 5 Déploiement du système IDS/IPS Suricata

L'objectif de cette phase est la mise en uvre de Suricata sur le pare-feu pfSense afin d'assurer les fonctions de détection (IDS) et de prévention (IPS) des intrusions sur l'interface WAN. Ce déploiement permet de surveiller le trafic à la frontière du réseau et de bloquer les comportements malveillants simulés depuis la machine attaquante Kali Linux.

9.1 Étape 1 Installation du paquet Suricata

Le déploiement commence par l'intégration du paquet binaire au sein du système pfSense via le gestionnaire de paquets centralisé (*Package Manager*). Après récupération depuis les dépôts officiels, le service est installé de manière native.

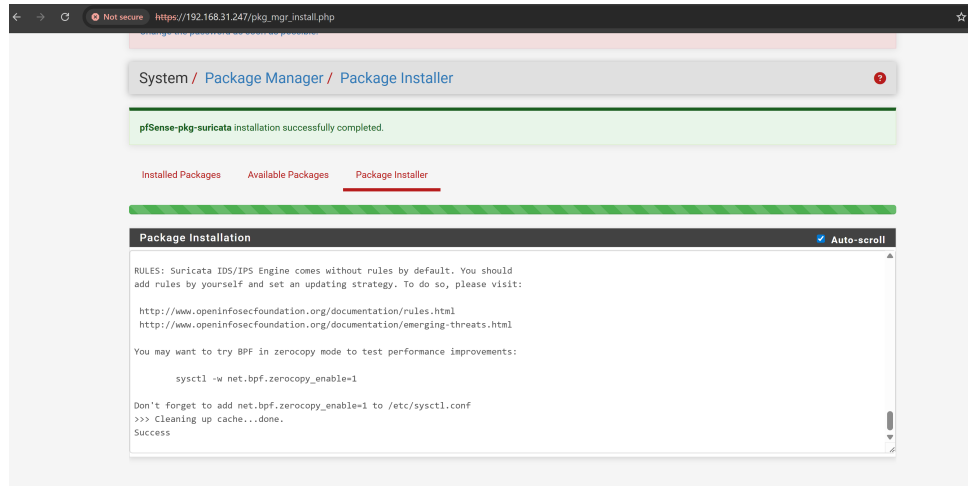


FIGURE 58 – Installation et validation du package Suricata avec succès sur pfSense.

9.2 Étape 2 Configuration globale du moteur

L'étape suivante consiste à définir les paramètres globaux d'exécution du moteur d'inspection. L'activation des signatures de la communauté *Emerging Threats (ET) Open* est réalisée, accompagnée d'une automatisation du cycle de mise à jour (toutes les 12 heures) et d'un mécanisme de purge automatique des hôtes bloqués après 15 minutes.

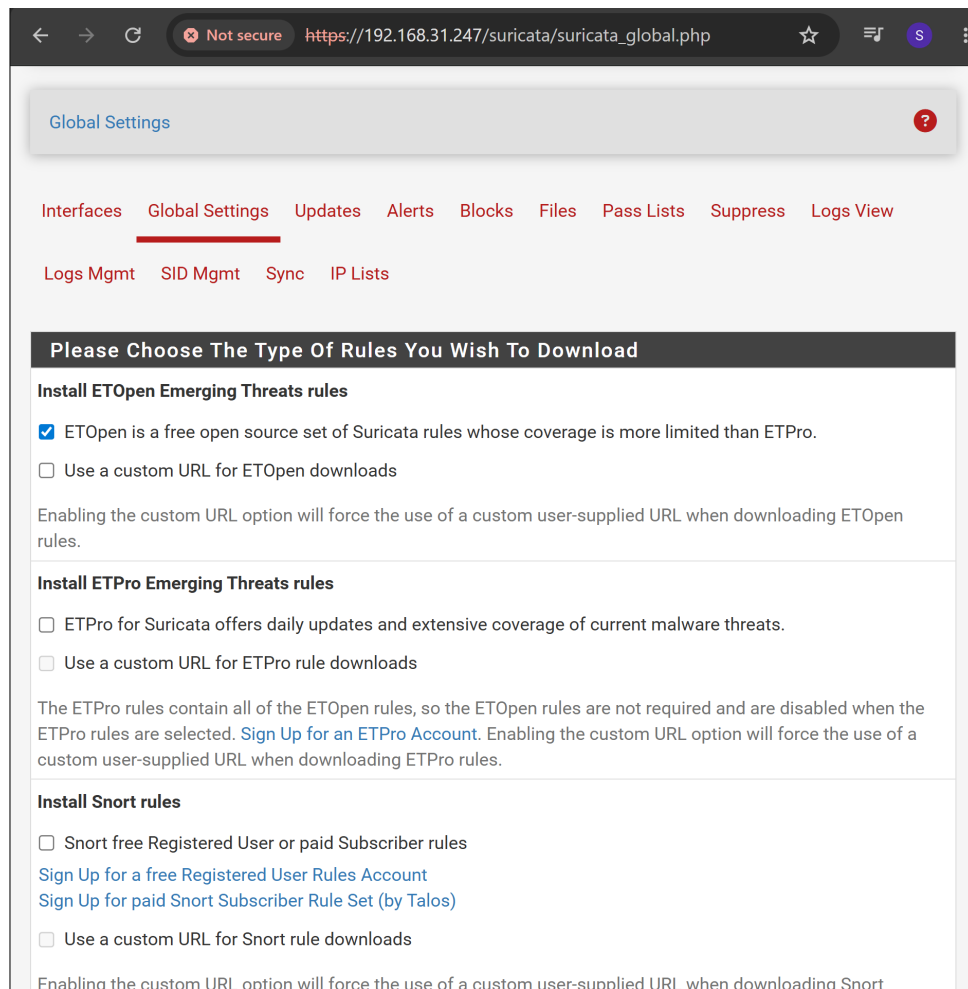


FIGURE 59 – Configuration globale et initialisation des flux de règles.

9.3 Étape 3 Synchronisation et mise à jour des signatures

Avant toute mise en production, une synchronisation manuelle est opérée pour télécharger la base de données des signatures de menaces (fichiers de règles *.rules*). Cela garantit que le moteur dispose des derniers indicateurs de compromission (IoC) connus.

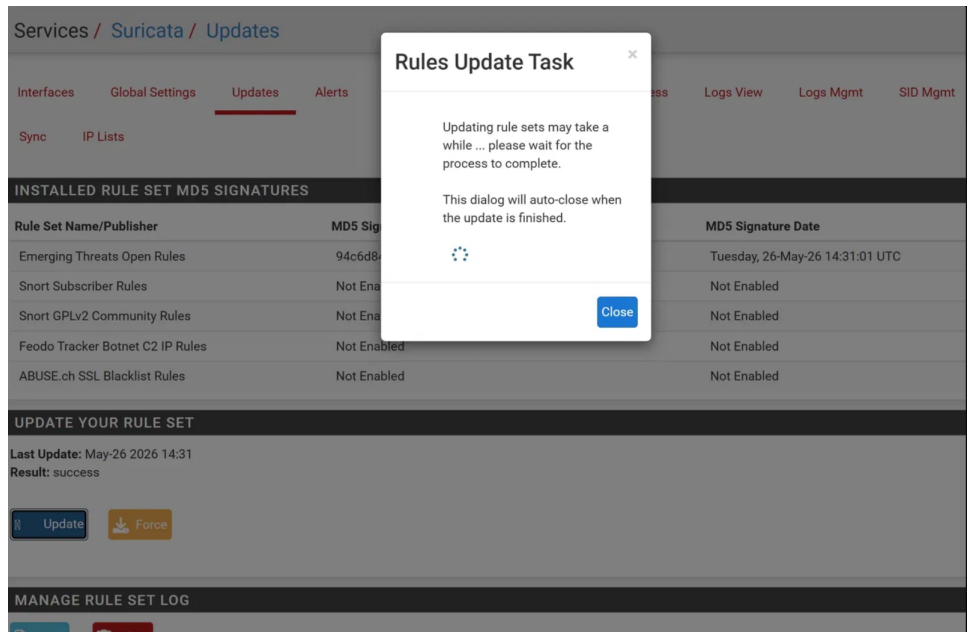


FIGURE 60 – Mise à jour et téléchargement complet de la base de règles.

9.4 Étape 4 Configuration de l'interface WAN

L'instance d'inspection est affectée à l'interface WAN. Les paramètres d'alerte sont durcis en activant le mode **Block Offenders** couplé à l'option **Kill States** afin d'interrompre instantanément toutes les connexions actives d'une adresse IP identifiée comme malveillante. Le mode d'exécution choisi est *Workers* avec un profil de performance moyen.

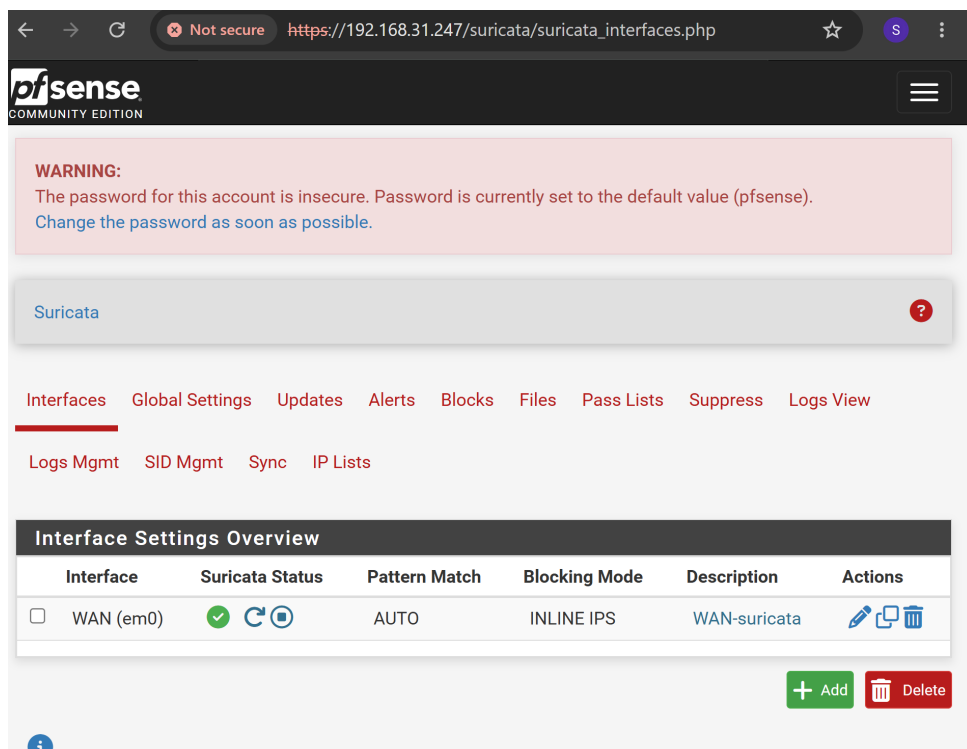


FIGURE 61 – Configuration des paramètres de blocage (*IPS inline*) sur l'interface WAN.

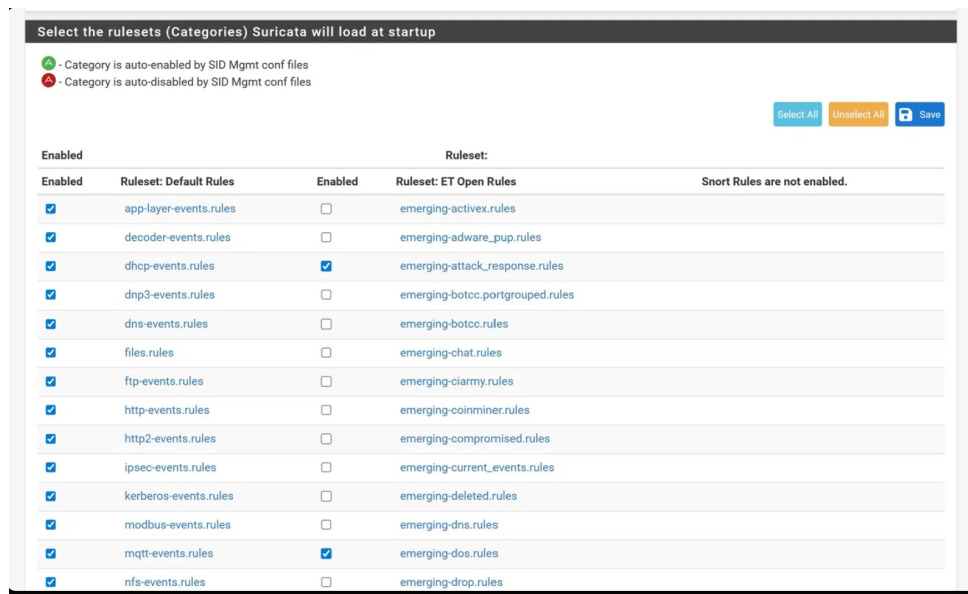


FIGURE 62 – .Sélection et activation des catégories Emerging Threats requises

9.5 Étape 5 Sélection des catégories de règles de détection

Parmi le catalogue de règles téléchargé, les ensembles spécifiques à notre contexte de test et de conformité sont activés (scans, dénis de service, vulnérabilités courantes, attaques de force brute et anomalies ICMP).

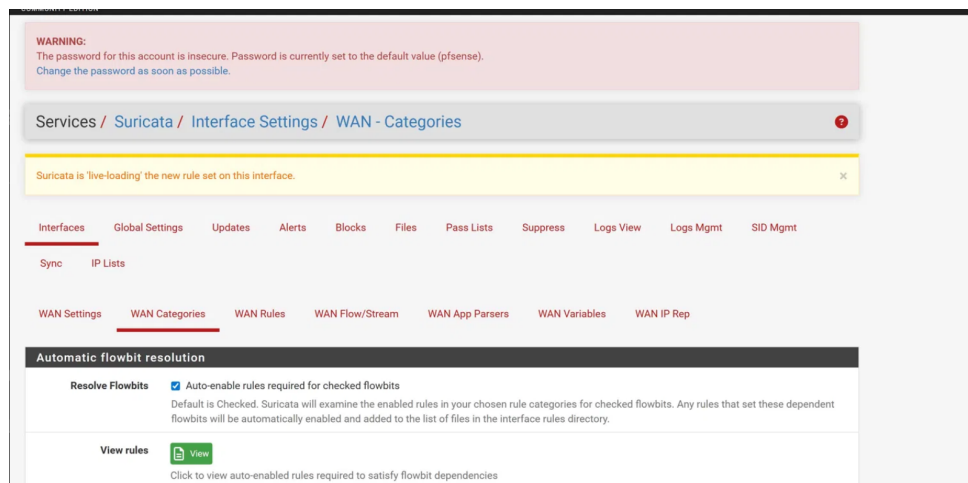


FIGURE 63 – Validation de l'affectation de l'interface et des flux associés.

9.6 Étape 6 Implémentation des règles personnalisées (*Custom Rules*)

Afin de répondre précisément aux exigences du cahier des charges, quatre règles spécifiques hors-catalogue ont été injectées dans la section *custom.rules* de l'interface WAN. Ces signatures se focalisent sur la détection des activités de reconnaissance (*Nmap*) et des attaques applicatives ou de flux.

Listing 1 – Règles personnalisées injectées dans Suricata

```
1 # Scan SYN Nmap
2 alert tcp any any -> $HOME_NET any (msg:"SCAN SYN Nmap detected";
   flags:S; threshold:type both, track by_src, count 20, seconds
   3; classtype:network-scan; sid:9000001; rev:1;)
3
4 # Brute force SSH
5 alert tcp any any -> $HOME_NET 22 (msg:"BRUTEFORCE SSH attempt";
   flow:to_server; threshold:type both, track by_src, count 5,
   seconds 60; classtype:attempted-admin; sid:9000002; rev:1;)
6
7 # ICMP flood
8 alert icmp any any -> $HOME_NET any (msg:"ICMP FLOOD detected";
   threshold:type both, track by_src, count 50, seconds 10;
   classtype:denial-of-service; sid:9000003; rev:1;)
9
10 # Nmap service detection
11 alert tcp any any -> $HOME_NET any (msg:"SCAN Service Detection";
   content:"Nmap"; nocase; classtype:network-scan; sid:9000004;
   rev:1;)
```

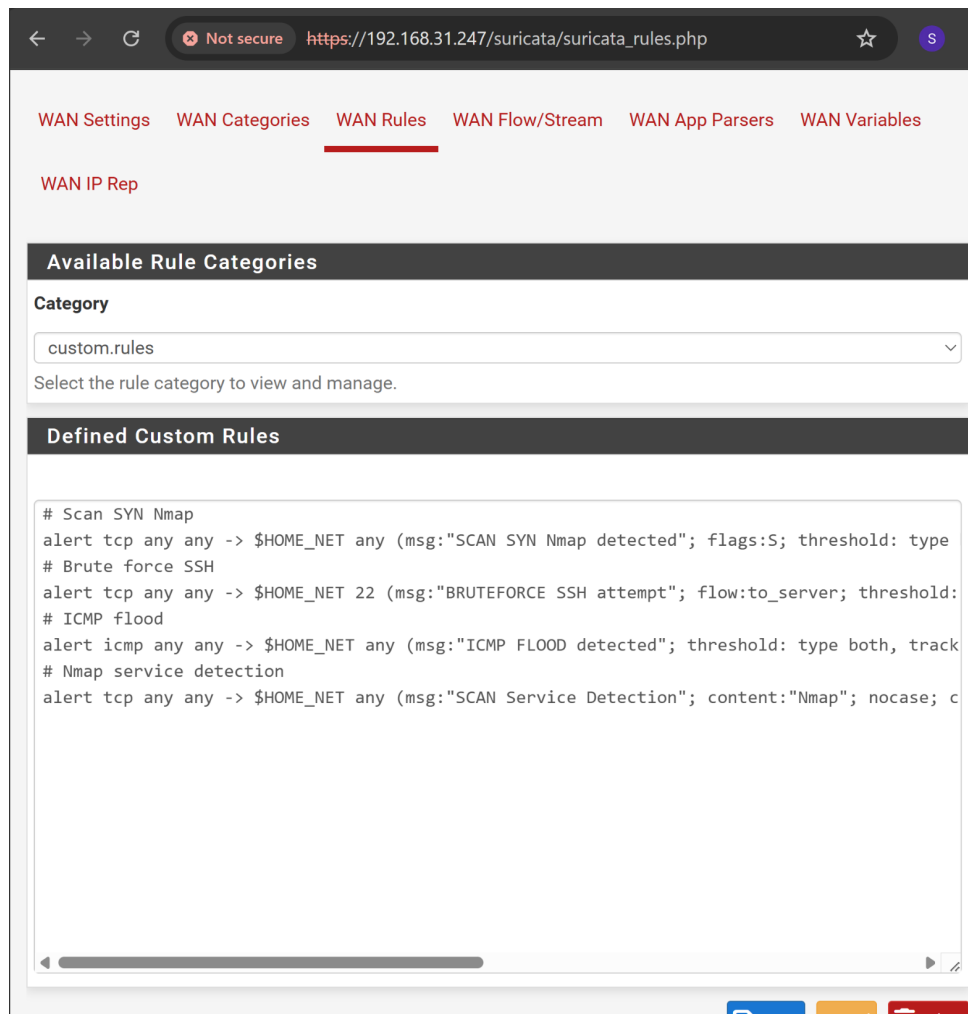


FIGURE 64 – Édition et enregistrement des règles locales personnalisées.

9.7 Étape 7 Initialisation du service et vérification

L'ensemble de la configuration étant appliqué, l'instance Suricata est démarrée sur l'interface réseau. Le tableau de bord valide l'état opérationnel nominal du service en affichant l'indicateur d'exécution actif.

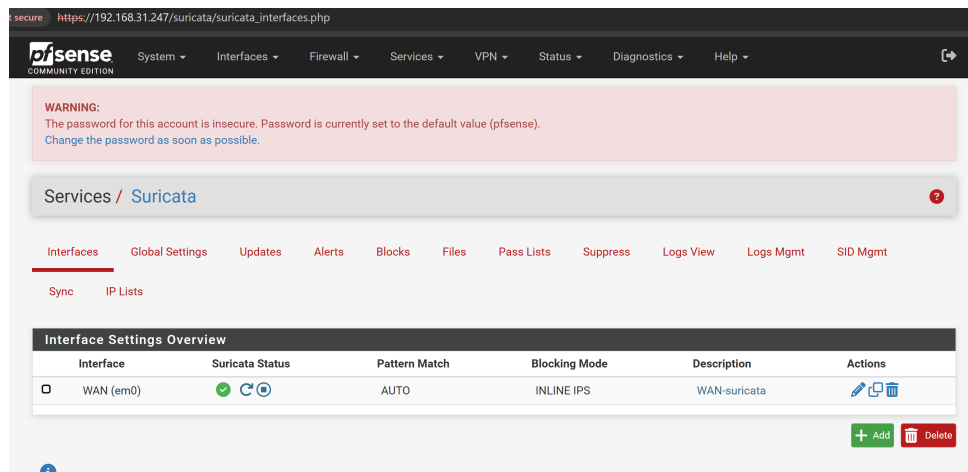


FIGURE 65 – Moteur Suricata actif et opérationnel (Statut : **RUNNING**) en mode INLINE IPS.

9.8 Étape 8 Observation de l'état initial des alertes

Une vérification finale de l'onglet *Alerts* est menée à blanc. Comme attendu, aucune signature n'est déclenchée en l'absence de trafic hostile, ce qui valide l'absence de faux positifs au repos avant le lancement de la phase offensive.

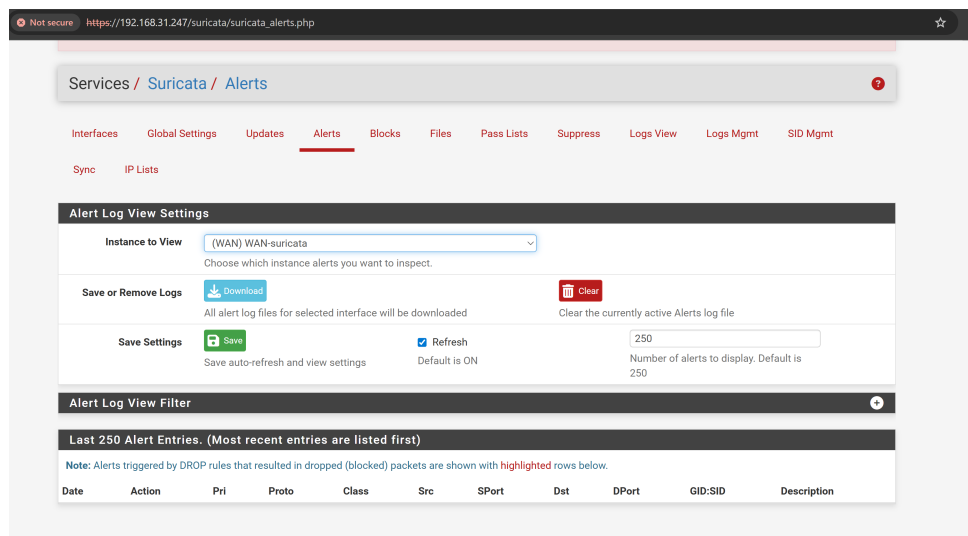


FIGURE 66 – Interface de supervision des alertes vierge avant le début des simulations d'attaques.

9.9 Bilan d'avancement de l'architecture

La configuration de la couche défensive périphérique est désormais entièrement finalisée. Le tableau ci-dessous valide l'état de complétion de cette phase :

Étape de configuration	Statut
Suricata installé	OK
Configuration globale (<i>Global Settings</i>)	OK
Mise à jour des règles	OK
Paramétrage de l'interface WAN	OK
Activation des catégories d'alertes	OK
Injectons des règles personnalisées (<i>Custom</i>)	OK
Démarrage du service (Mode <i>Inline IPS</i>)	RUNNING

TABLE 4 – Checklist de validation de la Partie 5.

10 Phase 6 Attaques depuis Kali Linux

10.1 Contexte

Cette phase simule des attaques réelles depuis une machine Kali Linux (192.168.31.90) vers le serveur Ubuntu (192.168.56.10) à travers le firewall pfSense. Une route statique a été configurée sur Kali pour atteindre le réseau LAN via pfSense :

```
sudo ip route add 192.168.56.0/24 via 192.168.31.247
```

10.2 Attaque 1 Scan réseau SYN (Nmap -sS)

Le scan SYN est une technique de reconnaissance permettant de détecter les ports ouverts sans établir de connexion complète.

```
sudo nmap -sS -Pn -T4 192.168.56.10
```

```

(kali@kali)-[~]
$ sudo ip route add 192.168.56.0/24 via 192.168.31.247

(kali@kali)-[~]
$ sudo nmap -sS -Pn -T4 192.168.56.10

Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-27 15:17 -0400
Nmap scan report for 192.168.56.10
Host is up.
All 1000 scanned ports on 192.168.56.10 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 107.77 seconds

(kali@kali)-[~]
$
    
```

FIGURE 67 – Scan réseau SYN depuis Kali Linux

Not secure https://192.168.31.247/suricata/suricata_alerts.php?instance=0

Save auto-refresh and view settings Default is ON Number of alerts to display: Default is 250

Alert Log View Filter

Last 250 Alert Entries. (Most recent entries are listed first)

Note: Alerts triggered by DROP rules that resulted in dropped (blocked) packets are shown with highlighted rows below.

Date	Action	Pri	Proto	Class	Src	SPort	Dst	DPort	GID/SID	Description
05/27/2026 19:19:07	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41908	192.168.56.10	100	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:19:05	⚠	2	TCP	Potentially Bad Traffic	192.168.31.90	41910	192.168.56.10	5432	1:2010939	ET SCAN Suspicious inbound to PostgreSQL port 5432
05/27/2026 19:19:05	⚠	2	TCP	Potentially Bad Traffic	192.168.31.90	41908	192.168.56.10	5432	1:2010939	ET SCAN Suspicious inbound to PostgreSQL port 5432
05/27/2026 19:19:04	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41908	192.168.56.10	8001	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:19:01	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41908	192.168.56.10	9998	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:18:58	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41908	192.168.56.10	8254	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:18:54	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41910	192.168.56.10	16993	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:18:51	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41910	192.168.56.10	668	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:18:48	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41910	192.168.56.10	146	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:18:44	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	41910	192.168.56.10	2608	1:9000001	SCAN SYN Nmap detected

FIGURE 68 – Alertes Suricata générées après le scan SYN règle custom SID :9000001

10.3 Attaque 2 Détection de services (Nmap -sV)

La détection de services permet d'identifier les versions des logiciels en cours d'exécution sur les ports ouverts.

```
sudo nmap -sV -Pn -T4 192.168.56.10
```

```

(kali@kali)-[~]
$ sudo nmap -sV -Pn -T4 192.168.56.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-27 15:23 -0400
Nmap scan report for 192.168.56.10
Host is up.
All 1000 scanned ports on 192.168.56.10 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 105.85 seconds

(kali@kali)-[~]
$
  
```

FIGURE 69 – Détection de services depuis Kali Linux

Not secure https://192.168.31.247/suricata/suricata_alerts.php?instance=0

Alert Log View Filter

Last 250 Alert Entries. (Most recent entries are listed first)

Note: Alerts triggered by DROP rules that resulted in dropped (blocked) packets are shown with highlighted rows below.

Date	Action	Pri	Proto	Class	Src	SPort	Dst	DPort	GID:SID	Description
05/27/2026 19:25:22	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36403	192.168.56.10	16001	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:19	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36403	192.168.56.10	3703	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:19	⚠	2	TCP	Attempted Information Leak	192.168.31.90	36401	192.168.56.10	5811	1:2002910	ET SCAN Potential VNC Scan 5800-5820
05/27/2026 19:25:16	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36403	192.168.56.10	55555	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:13	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36401	192.168.56.10	2200	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:10	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36401	192.168.56.10	7627	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:09	⚠	2	TCP	Potentially Bad Traffic	192.168.31.90	36403	192.168.56.10	1521	1:2010936	ET SCAN Suspicious inbound to Oracle SQL port 1521
05/27/2026 19:25:08	⚠	2	TCP	Potentially Bad Traffic	192.168.31.90	36401	192.168.56.10	1521	1:2010936	ET SCAN Suspicious inbound to Oracle SQL port 1521
05/27/2026 19:25:07	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36401	192.168.56.10	7800	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:04	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36401	192.168.56.10	4004	1:9000001	SCAN SYN Nmap detected
05/27/2026 19:25:01	⚠	3	TCP	Detection of a Network Scan	192.168.31.90	36401	192.168.56.10	65129	1:9000001	SCAN SYN Nmap detected

FIGURE 70 – Alertes Suricata générées après la détection de services règle SID :9000004

10.4 Attaque 3 Ping Flood ICMP

Le ping flood est une attaque par déni de service qui saturé la cible avec un grand nombre de paquets ICMP.

```
sudo ping -f 192.168.56.10
```

```

(kali@kali)-[~]
$ sudo ping -f 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
.^C
— 192.168.56.10 ping statistics —
5708 packets transmitted, 5707 received, 0.0175193% packet loss, time 21981ms
rtt min/avg/max/mdev = 1.754/3.989/259.717/9.922 ms, pipe 14, ipg/ewma 3.851/2.465 ms
(kali@kali)-[~]
$
    
```

FIGURE 71 – Ping flood ICMP depuis Kali Linux

Not secure https://192.168.31.247/suricata/suricata_alerts.php?instance=0

Last 250 Alert Entries. (Most recent entries are listed first)

Note: Alerts triggered by DROP rules that resulted in dropped (blocked) packets are shown with highlighted rows below.

Date	Action	Pri	Proto	Class	Src	SPort	Dst	DPort	GID:SID	Description
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:43	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX

FIGURE 72 – Alertes Suricata générées après le ping flood GPL ICMP PING *NIX détecté

10.5 Attaque 4 Brute Force SSH (Hydra)

L'attaque par force brute SSH tente de deviner les credentials SSH en utilisant un dictionnaire de mots de passe.

```
hydra -l salma -P /usr/share/wordlists/rockyou.txt \
ssh://192.168.56.10 -t 4
```

```
(kali㉿kali)-[~]
$ hydra -l salma -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.10 -t
4

Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-27 15:
32:02
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.10:22/
[ERROR] could not connect to ssh://192.168.56.10:22 - Timeout connecting to 1
92.168.56.10

(kali㉿kali)-[~]
$
```

FIGURE 73 – Attaque brute force SSH depuis Kali Linux

Not secure https://192.168.31.247/suricata/suricata_alerts.php?instance=0

Alert Log View Filter

Last 250 Alert Entries. (Most recent entries are listed first)

Note: Alerts triggered by DROP rules that resulted in dropped (blocked) packets are shown with highlighted rows below.

Date	Action	Pri	Proto	Class	Src	SPort	Dst	DPort	GID:SID	Description
05/27/2026 19:32:13	⚠	2	TCP	Attempted Information Leak	192.168.31.90	45840	192.168.56.10	22	1:2001219	ET SCAN Potential SSH Scan
05/27/2026 19:32:13	⚠	1	TCP	Attempted Administrator Privilege Gain	192.168.31.90	45840	192.168.56.10	22	1:9000002	BRUTEFORCE SSH attempt
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX
05/27/2026 19:29:50	⚠	3	ICMP	Misc activity	192.168.31.90	8	192.168.56.10	0	1:2100366	GPL ICMP PING *NIX

FIGURE 74 – Alertes Suricata générées après le brute force SSH règle custom SID :9000002 et ET SCAN Potential SSH Scan

10.6 Bilan des attaques détectées

Attaque	Outil	SID	Statut
Scan SYN	Nmap -sS	9000001	Détecté
Détection services	Nmap -sV	9000004	Détecté
Ping Flood	ping -f	2100366	Détecté
Brute Force SSH	Hydra	9000002	Détecté

TABLE 5 – Récapitulatif des attaques et détections Suricata

11 Phase 7 Firewalling

11.1 Objectif

Cette phase consiste à configurer des règles de filtrage sur pfSense afin de contrôler et restreindre le trafic réseau selon la politique de sécurité définie. Quatre règles principales ont été mises en place.

11.2 Règle 1 Blocage ICMP sur WAN

La première règle bloque tout trafic ICMP entrant sur l'interface WAN, empêchant ainsi les tentatives de reconnaissance réseau (ping) depuis l'extérieur vers le firewall pf-Sense.

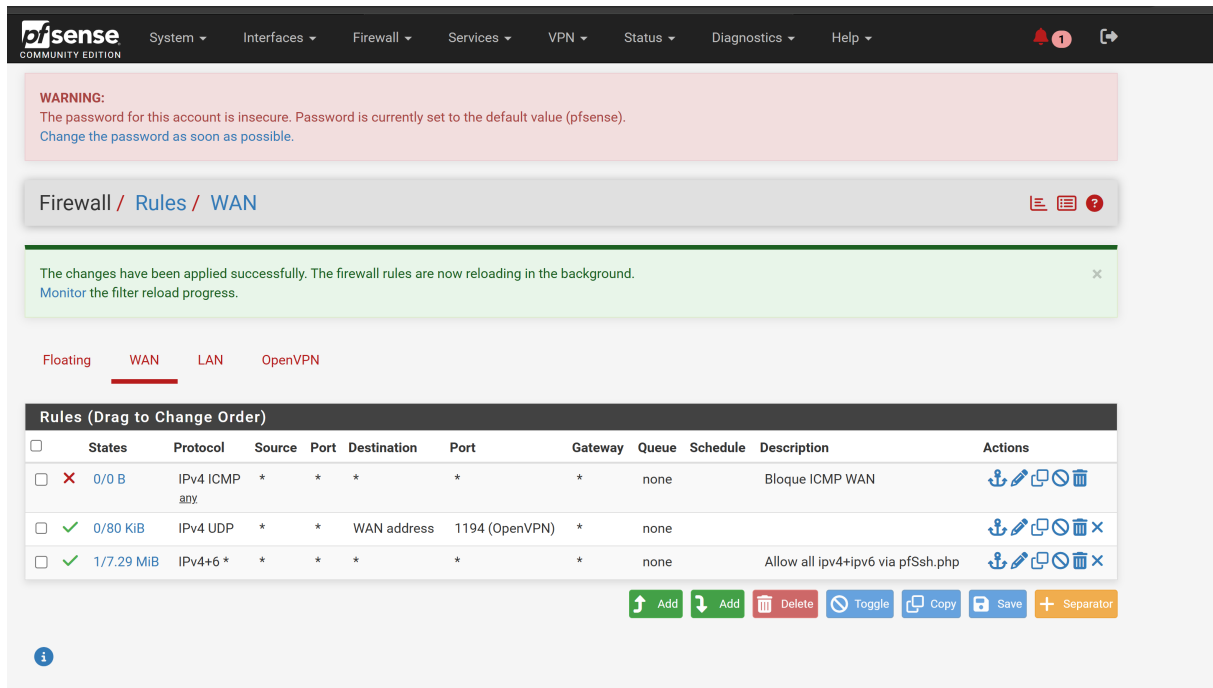


FIGURE 75 – Règle de blocage ICMP sur l'interface WAN

11.3 Règle 2 Autoriser HTTP via VPN uniquement

Cette règle autorise uniquement le trafic HTTP (port 80) provenant du réseau VPN (10.8.0.0/24) vers le serveur Ubuntu (192.168.56.0/24). Tout accès HTTP depuis l'extérieur sans VPN est refusé.

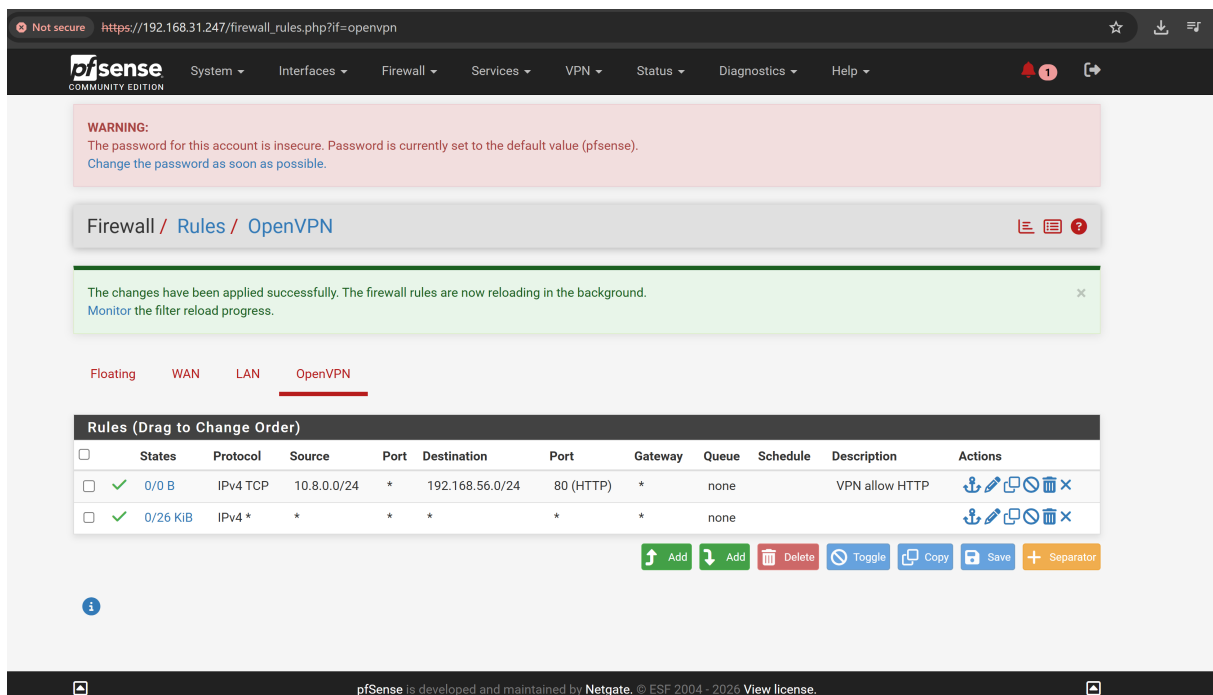


FIGURE 76 – Règle autorisant HTTP depuis le réseau VPN vers le serveur Ubuntu

11.4 Règle 3 Autoriser SSH via VPN uniquement

Cette règle restreint l'accès SSH (port 22) au serveur Ubuntu exclusivement aux clients connectés via le VPN (10.8.0.0/24). Toute tentative de connexion SSH depuis un réseau non-VPN est bloquée, renforçant ainsi la sécurité de l'administration distante.

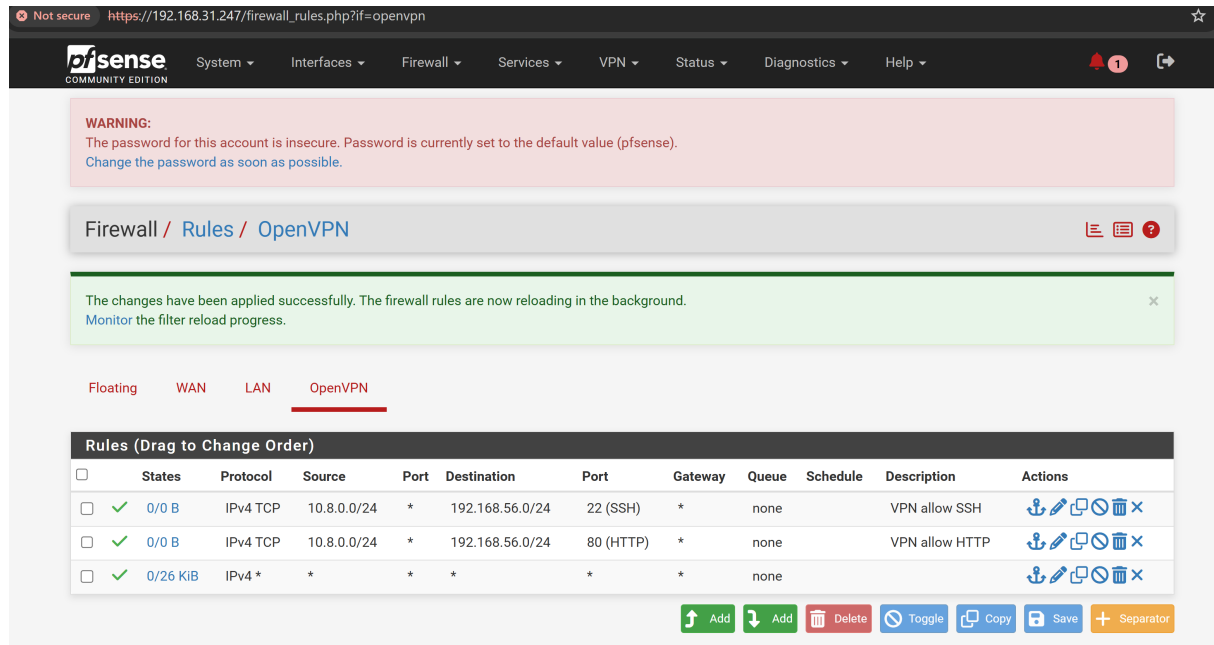


FIGURE 77 – Règle autorisant SSH uniquement via le tunnel VPN

11.5 Règle 4 Blocage total de la machine Kali

Suite aux attaques détectées par Suricata (scans Nmap, brute force SSH, ping flood), une règle de blocage total a été ajoutée sur l'interface WAN pour bloquer toute communication provenant de la machine Kali Linux (192.168.31.90).

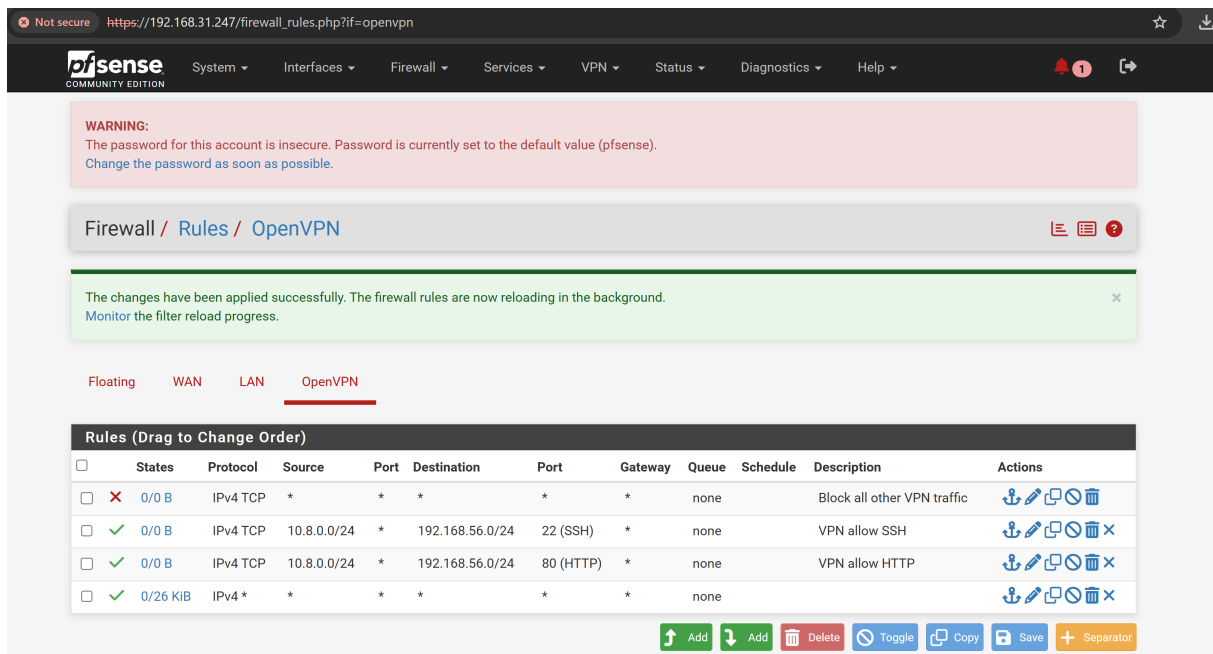
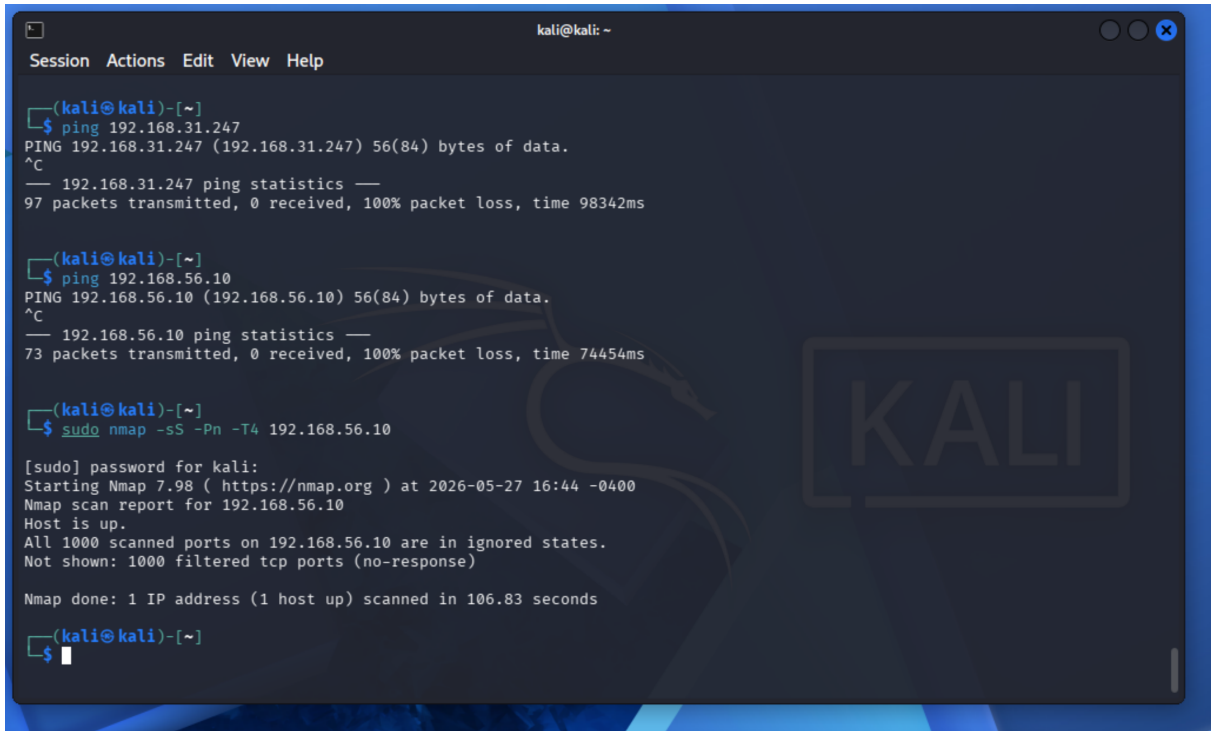


FIGURE 78 – Règle de blocage total de la machine attaquante Kali Linux

11.6 Tests de validation

11.6.1 Test 1 Kali Linux bloquée

Après application des règles firewall, toute tentative de communication depuis Kali Linux vers le réseau interne est bloquée. Les scans Nmap et les pings ne reçoivent aucune réponse, confirmant l'efficacité de la règle de blocage.



```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ ping 192.168.31.247
PING 192.168.31.247 (192.168.31.247) 56(84) bytes of data.
^C
--- 192.168.31.247 ping statistics ---
97 packets transmitted, 0 received, 100% packet loss, time 98342ms

(kali@kali)-[~]
$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
^C
--- 192.168.56.10 ping statistics ---
73 packets transmitted, 0 received, 100% packet loss, time 74454ms

(kali@kali)-[~]
$ sudo nmap -sS -Pn -T4 192.168.56.10

[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-27 16:44 -0400
Nmap scan report for 192.168.56.10
Host is up.
All 1000 scanned ports on 192.168.56.10 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

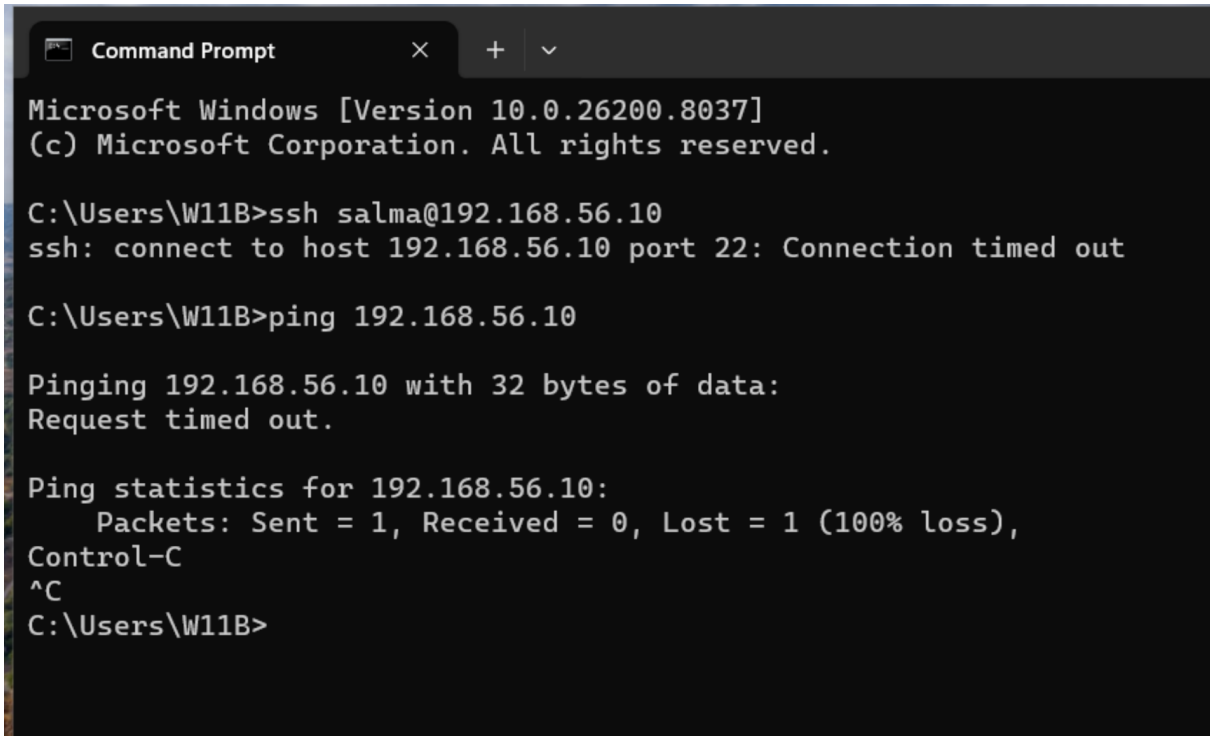
Nmap done: 1 IP address (1 host up) scanned in 106.83 seconds

(kali@kali)-[~]
$
  
```

FIGURE 79 – Confirmation du blocage total de Kali Linux après application des règles firewall

11.6.2 Test 2 Accès refusé sans VPN

Depuis le poste Windows sans connexion VPN active, toute tentative d'accès SSH ou HTTP vers le serveur Ubuntu (192.168.56.10) est bloquée par le firewall, conformément à la politique de sécurité définie.



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\W11B>ssh salma@192.168.56.10
ssh: connect to host 192.168.56.10 port 22: Connection timed out

C:\Users\W11B>ping 192.168.56.10

Pinging 192.168.56.10 with 32 bytes of data:
Request timed out.

Ping statistics for 192.168.56.10:
    Packets: Sent = 1, Received = 0, Lost = 1 (100% loss),
Control-C
^C
C:\Users\W11B>
```

FIGURE 80 – Accès SSH et HTTP refusé depuis Windows sans connexion VPN

11.6.3 Test 3 Accès autorisé avec VPN

Une fois le VPN connecté, le poste Windows obtient une adresse dans le réseau 10.8.0.0/24 et peut accéder au serveur Ubuntu via SSH (port 22) et HTTP (port 80), conformément aux règles OpenVPN configurées. La page Apache2 est accessible et la connexion SSH est établie avec succès.

```

Command Prompt

Swap usage: 0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how Mic
roK8s
just raised the bar for easy, resilient and secure K8s cluster deploy
ment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Che
ck your Internet connection or proxy settings

Last login: Wed May 27 20:03:51 2026 from 10.8.0.2
salma@salma:~$ exit
logout
Connection to 192.168.56.10 closed.
    
```

FIGURE 81 – Accès SSH, ping et Apache2 autorisés depuis Windows après connexion au VPN

11.7 Bilan du Firewalling

Règle	Interface	Action	Résultat
Bloquer ICMP	WAN	Block	Ping WAN refusé
Autoriser HTTP	OpenVPN	Pass port 80	HTTP via VPN
Autoriser SSH	OpenVPN	Pass port 22	SSH via VPN
Bloquer Kali	WAN	Block	Kali isolée

TABLE 6 – Récapitulatif des règles firewall et résultats des tests

12 Conclusion

Ce travail pratique nous a permis de concevoir et déployer une architecture réseau sécurisée complète en environnement virtualisé, en intégrant plusieurs couches de sécurité complémentaires.

12.1 Bilan des réalisations

Au terme de ce laboratoire, nous avons mis en place une infrastructure composée de quatre machines virtuelles interconnectées, simulant un environnement d'entreprise

réaliste avec ses menaces et ses mécanismes de défense.

- **Architecture réseau** : Une topologie sécurisée a été conçue avec pfSense comme point central de contrôle, séparant le réseau interne (Host-Only) du réseau externe (Bridged), garantissant ainsi l'isolation du serveur critique Ubuntu.
- **VPN Client-to-Site** : Le déploiement d'OpenVPN sur pfSense a permis d'établir un tunnel chiffré entre le client Windows distant et le réseau interne, assurant la confidentialité et l'intégrité des communications distantes.
- **Durcissement du serveur Ubuntu** : La sécurisation du serveur Linux a été réalisée à plusieurs niveaux : désactivation d'IPv6, configuration des paramètres sysctl, activation du firewall UFW, sécurisation de PAM et restriction des accès SSH par authentification par clés.
- **Déploiement IDS/IPS Suricata** : L'installation et la configuration de Suricata en mode Inline IPS sur pfSense ont permis la détection et le blocage en temps réel des menaces, grâce aux règles Emerging Threats Open et aux règles personnalisées développées spécifiquement pour ce laboratoire.
- **Simulation d'attaques** : Les attaques simulées depuis Kali Linux (scan SYN Nmap, détection de services, ping flood ICMP et brute force SSH via Hydra) ont toutes été détectées avec succès par Suricata, validant l'efficacité du système de détection.
- **Firewalling** : Les règles de filtrage configurées sur pfSense ont permis de restreindre les accès selon le principe du moindre privilège : blocage ICMP sur WAN, restriction SSH et HTTP exclusivement via VPN, et isolation complète de la machine attaquante.

12.2 Compétences acquises

Ce laboratoire a permis de développer des compétences pratiques dans plusieurs domaines de la cybersécurité :

- Administration et configuration de firewall pfSense
- Mise en place et gestion de VPN OpenVPN
- Durcissement de systèmes Linux selon les bonnes pratiques
- Déploiement et configuration d'un IDS/IPS avec Suricata
- Simulation d'attaques réelles avec les outils Kali Linux (Nmap, Hydra)
- Analyse et interprétation des logs de sécurité
- Configuration de règles firewall avancées

12.3 Enseignements et perspectives

Ce TP illustre l'importance d'une approche de sécurité en profondeur (*Defense in Depth*), où chaque couche de protection renforce les autres. Aucune solution unique ne suffit à elle seule : la combinaison VPN + IDS/IPS + Firewall + Durcissement système constitue une architecture robuste face aux menaces modernes.

En perspective, cette infrastructure pourrait être enrichie par :

- L'ajout d'un SIEM (Security Information and Event Management) pour centraliser les logs
- La mise en place d'une authentification multi-facteurs (MFA) pour le VPN
- Le déploiement d'un serveur de supervision comme Zabbix ou Nagios

— L’automatisation de la réponse aux incidents via des scripts de blocage dynamique

En conclusion, ce laboratoire constitue une expérience formatrice qui reflète les défis réels auxquels font face les ingénieurs en sécurité informatique, en combinant théorie et pratique dans un environnement contrôlé et reproductible.